

ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ



NGUYỄN CHÍ NGHĨA

KIỂM THỬ REST API BẰNG HỆ ĐA TÁC TỬ
KẾT HỢP PHÂN TÍCH TĨNH VÀ
VÒNG LẶP CHUYÊN GIA

KHOÁ LUẬN TỐT NGHIỆP

Ngành: Khoa học máy tính

HÀ NỘI - 2026

ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ



NGUYỄN CHÍ NGHĨA

KIỂM THỬ REST API BẰNG HỆ ĐA TÁC TỬ
KẾT HỢP PHÂN TÍCH TĨNH VÀ
VÒNG LẶP CHUYÊN GIA

KHOÁ LUẬN TỐT NGHIỆP

Ngành: Khoa học máy tính

Cán bộ hướng dẫn: PGS.TS. Võ Đình Hiếu

HÀ NỘI - 2026

Lời cảm ơn

Em xin chân thành cảm ơn tới thầy PGS.TS Võ Đình Hiếu đã tận tình hướng dẫn, định hướng và hỗ trợ em trong suốt quá trình thực hiện khóa luận tốt nghiệp. Những góp ý chuyên môn và sự chỉ dẫn của thầy đã giúp em hoàn thiện nội dung nghiên cứu cũng như phương pháp thực hiện đề tài.

Em xin chân thành cảm ơn.

Lời cam đoan

Tôi là Nguyễn Chí Nghĩa, sinh viên lớp K67I-CS3. Tôi xin cam đoan rằng khóa luận tốt nghiệp này là kết quả nghiên cứu và thực hiện của riêng tôi dưới sự hướng dẫn của giảng viên hướng dẫn.

Các nội dung, kết quả và số liệu trình bày trong khóa luận là trung thực và chưa từng được công bố trong bất kỳ công trình nào khác. Những tài liệu, kết quả nghiên cứu của các tác giả khác được sử dụng trong khóa luận đều đã được trích dẫn rõ ràng theo đúng quy định.

Nếu có bất kỳ sai sót hoặc vi phạm nào liên quan đến nội dung cam đoan trên, tôi xin hoàn toàn chịu trách nhiệm theo quy định của Nhà trường.

Hà Nội, ngày 31 tháng 05 năm 2026

Sinh viên

Nguyễn Chí Nghĩa

Tóm tắt

REST API đóng vai trò nền tảng trong thiết lập giao thức tương tác và trao đổi dữ liệu giữa các hệ thống phần mềm phân tán nhờ tính chất phi trạng thái và khả năng mở rộng. Tuy nhiên, việc đảm bảo chất lượng hệ thống đối mặt thách thức lớn từ sự gia tăng quy mô và độ phức tạp của quy tắc nghiệp vụ. Các phương pháp kiểm thử API hiện nay từ kiểm thử mờ (*fuzzing*) đến các công cụ dựa trên đa tác tử AI chủ yếu tập trung vào việc phát hiện lỗi hệ thống (*server crash*) và tăng độ bao phủ mã nguồn, nhưng gặp hạn chế trong việc kiểm chứng logic nghiệp vụ và có tỉ lệ dương tính giả (*false positive*) cao. Hạn chế này đến từ hai điểm nghẽn kiến trúc cốt lõi: (1) Vi phạm ràng buộc tĩnh do chiết xuất tài liệu kém, gây lỗi 400 nhưng bị nhầm là lỗi logic; và (2) Cơ chế bộ nhớ phẳng (*flat memory*) khiến tác tử bị ảo giác (*hallucination*) và lẫn lộn nhận thức nghiệp vụ. Khóa luận đề xuất kiến trúc kiểm thử đa tác tử lai ghép phân tích tĩnh và suy luận động. Hệ thống ứng dụng cơ chế Quan sát - Xác nhận (*Observation - Confirmation*) từ *RBCTest* để chiết xuất các ràng buộc cứng từ OpenAPI nhằm sinh yêu cầu hợp lệ và loại bỏ lỗi cú pháp. Đồng thời, tích hợp bộ nhớ học tập *Hindsight* kết hợp vòng lặp chuyên gia, cho phép tác tử lưu vết kinh nghiệm thực thi, cập nhật hệ thống niềm tin và giảm thiểu chuỗi ảo giác. Thực nghiệm trên 10 hệ thống REST API thực tế cho thấy khả năng phát hiện chính xác các lỗi logic, giảm tỷ lệ dương tính giả xuống từ 38% ban đầu xuống dưới 15%.

Từ khóa: Kiểm thử REST API, Hệ đa tác tử, Logic nghiệp vụ, Bộ nhớ Hindsight, Quan sát - Xác nhận, Dương tính giả.

Mục lục

Lời cảm ơn

Lời cam đoan i

Tóm tắt ii

Mục lục iii

Danh sách hình vẽ vi

Danh sách bảng vii

Danh mục các từ viết tắt viii

Chương 1 Đặt vấn đề 1

1.1 Bối cảnh 1

1.2 Mô tả bài toán 2

1.3 Mục tiêu và phạm vi nghiên cứu 3

1.4 Cấu trúc khóa luận 3

Chương 2 Cơ sở lý thuyết 5

2.1 Kiến trúc REST, REST API và Đặc tả OpenAPI 5

2.1.1 Kiến trúc REST 5

2.1.2 REST API 5

2.1.3 Đặc tả OpenAPI 6

2.2 Kiểm thử logic trong REST API 8

2.2.1 Kiểm thử mờ 8

2.2.2 Kiểm thử logic 8

2.3 Mô hình ngôn ngữ lớn 9

2.3.1 Mô hình ngôn ngữ lớn 9

2.3.2 Hạn chế của Mô hình ngôn ngữ lớn 9

2.3.3	Kỹ thuật Prompting	10
2.4	Kiến trúc Hệ đa tác tử dựa trên LLM	10
2.4.1	Kiến trúc Hệ đa tác tử	10
2.4.2	Bộ nhớ ngắn hạn và dài hạn	11
2.5	Kiến trúc bộ nhớ cấu trúc	12
2.5.1	Hạn chế của kiến trúc RAG	12
2.5.2	Kiến trúc bộ nhớ Hindsight	12
2.6	Các nghiên cứu liên quan	13
2.6.1	LogiAgent: Hệ đa tác tử dựa trên LLM cho kiểm thử logic REST API	13
2.6.2	RBCTest: Khai phá và xác minh oracle phản hồi API REST bằng mô hình ngôn ngữ lớn	14
2.6.3	Hindsight: Kiến trúc bộ nhớ cho tác tử học như con người	14
2.6.4	Khoảng trống nghiên cứu	15
Chương 3	Phương pháp đề xuất	16
3.1	Tổng quan phương pháp đề xuất	16
3.2	Kiến trúc kiểm thử kế thừa từ LogiAgent	17
3.2.1	API Relationship Graph	17
3.2.2	Luồng hoạt động đa tác tử	18
3.3	Cơ chế Observation–Confirmation cho xác thực ràng buộc tĩnh	19
3.3.1	Pha Observation	19
3.3.2	Pha Confirmation	20
3.3.3	Sinh và xác thực verification script	21
3.3.4	Tích hợp vào vòng kiểm thử	23
3.4	Cơ chế tương tác người dùng (Human-in-the-loop)	23
3.5	Tích hợp kiến trúc bộ nhớ Hindsight	25
3.5.1	Ánh xạ tri thức kiểm thử vào Hindsight	26
3.5.2	Cơ chế Retain	27
3.5.3	Cơ chế Recall	29
3.5.4	Reflection và cập nhật tri thức	30
3.6	Phân tích ưu điểm và hạn chế	31
3.6.1	Ưu điểm	31
3.6.2	Hạn chế	32

Chương 4 Thực nghiệm và đánh giá	34
4.1 Mục tiêu thực nghiệm	34
4.2 Thiết lập thực nghiệm	34
4.2.1 Dịch vụ thực nghiệm	34
4.2.2 Môi trường thực nghiệm	35
4.3 Phương pháp đối sánh	36
4.4 Kết quả thực nghiệm	37
4.4.1 RQ1: Khả năng phát hiện lỗi	37
4.4.2 RQ2: Khả năng kiểm soát False positive	38
4.4.3 RQ3: Đóng góp của từng thành phần	38
4.4.4 RQ4: Độ bao phủ kiểm thử	38
4.5 Phân tích và thảo luận	39
Kết luận	40
Tài liệu tham khảo	41

Danh sách hình vẽ

2.1	Lược đồ yêu cầu của API /annotation/genomic trong OAS Genome Nexus	6
2.2	Lược đồ đối tượng Genomic Location trong OAS Genome Nexus . .	7
2.3	Kiến trúc bộ nhớ Hindsight và các cơ chế Retain, Recall, Reflect . .	12
3.1	Kiến trúc tổng thể của hệ thống ARGOS	16
3.2	Quy trình Observation–Confirmation trong ARGOS	21
3.3	Quy trình kiểm chứng script với ví dụ	23
3.4	Giao diện control plane của Hindsight	25
3.5	Quy trình Retain trong ARGOS	28
3.6	Cơ chế multi-strategy Recall trong ARGOS	29
3.7	Quy trình Reflection và cập nhật tri thức trong ARGOS	31

Danh sách bảng

3.1	Ảnh xạ tri thức kiểm thử REST API vào các phân vùng bộ nhớ Hindsight	26
4.1	Các dịch vụ sử dụng cho đánh giá	35
4.2	Môi trường thực nghiệm của ARGOS	36
4.3	Thống kê số issue	37
4.4	Tỉ lệ FP trung bình của mỗi phương pháp	38

Danh mục các từ viết tắt

Từ viết tắt	Cụm từ đầy đủ	Ý nghĩa / Mô tả
API	Application Programming Interface	Giao diện lập trình ứng dụng
REST	Representational State Transfer	Chuyển đổi trạng thái đại diện
URI	Uniform Resource Identifier	Định danh tài nguyên thống nhất
OAS	OpenAPI Specification	Đặc tả OpenAPI
SUT	System Under Test	Hệ thống đang được kiểm thử
LLM	Large Language Model	Mô hình ngôn ngữ lớn
MAS	Multi Agent System	Hệ đa tác tử
FP	False Positive	Dương tính giả

Chương 1

Đặt vấn đề

1.1 Bối cảnh

Trong kỷ nguyên kiến trúc vi dịch vụ và điện toán đám mây, REST API đã trở thành thành phần cốt lõi trong việc thiết lập giao thức tương tác giữa các tầng hệ thống. Tuy nhiên, khi số lượng API gia tăng và mối liên kết giữa chúng ngày càng chặt chẽ, các quy tắc nghiệp vụ cũng trở nên phức tạp hơn. Do đó, việc đảm bảo chất lượng phần mềm không còn có thể chỉ dựa vào kiểm thử độc lập từng endpoint. Các phương pháp kiểm thử truyền thống như fuzzing hoặc kiểm thử đơn vị chủ yếu tập trung vào việc phát hiện các lỗi hệ thống nghiêm trọng (ví dụ: Server Error 5xx, lỗi định dạng dữ liệu), mà thường bỏ sót những sai lệch tinh vi trong logic nghiệp vụ có trạng thái (stateful). Theo phân tích từ Golmohammadi và cộng sự [1], các phương pháp kiểm thử định chuẩn dựa trên mã phản hồi HTTP chỉ xác nhận sự thành công ở mức độ giao thức truyền thông, nhưng không thể bảo đảm tính toàn vẹn của các quy trình xử lý nghiệp vụ bên trong. Đồng thuận với nhận định này, tài liệu chuyên môn từ mabl cùng các báo cáo bảo mật của Traceable [2] và OWASP [3] đều chỉ ra rằng khi phần lớn logic cốt lõi đã được chuyển dịch lên tầng API, việc dịch chuyển trọng tâm sang kiểm thử trực tiếp logic nghiệp vụ là yêu cầu cấp thiết nhằm khắc phục “điểm mù” của các công cụ truyền thống, từ đó ngăn chặn nguy cơ tin tặc khai thác các lỗ hổng và sai sót ở tầng sâu của hệ thống.

Sự trỗi dậy của các hệ thống tác tử trí tuệ nhân tạo dựa trên mô hình ngôn ngữ lớn (LLM-based agents) đã mở ra một hướng tiếp cận mới cho tự động hóa kiểm thử logic nghiệp vụ. Tuy nhiên, các nghiên cứu tiên phong như LogiAgent [4] vẫn đang đối mặt với thách thức lớn về độ tin cậy: kết quả thực nghiệm cho thấy tỷ lệ dương tính giả có thể lên tới 38.2%, một con số đáng kể gây lãng phí nguồn lực trong khâu thẩm định thủ công. Nguyên nhân cốt lõi xuất phát từ hai điểm nghẽn kiến trúc. Thứ nhất, đặc tả OpenAPI (OAS) — nguồn mô tả chuẩn cho cấu trúc và hành vi của hệ thống — chưa được tận dụng đầy đủ để suy luận và kiểm tra

trước các ràng buộc tĩnh. Vì vậy, tác tử thường sinh ra các yêu cầu không hợp lệ (lỗi 400) nhưng lại nhầm lẫn chúng với lỗi logic của hệ thống. Thứ hai, tri thức hiện được lưu trữ theo dạng bộ nhớ phẳng thiếu cấu trúc hợp lý. Kết hợp với cơ chế truy xuất thông tin dựa trên BM25 — đối sánh từ khóa đơn thuần, điều này khiến tác tử dễ nhầm lẫn ngữ cảnh, suy diễn sai quy tắc nghiệp vụ, và khó học hỏi một cách có hệ thống từ các thất bại trong các phiên kiểm thử trước. Do đó, việc nghiên cứu một giải pháp lai ghép, kết hợp khả năng khai phá ràng buộc tĩnh từ OAS với một kiến trúc bộ nhớ có khả năng tự tiến hóa nhận thức, là bài toán mang tính cấp thiết nhằm nâng cao độ tin cậy của tự động hóa kiểm thử phần mềm hiện đại.

1.2 Mô tả bài toán

Bài toán được nghiên cứu trong khóa luận là tự động phát hiện lỗi logic nghiệp vụ trong các hệ thống REST API có trạng thái thông qua kiến trúc đa tác tử sử dụng LLM. Mục tiêu của hệ thống là xây dựng và thực thi các chuỗi tương tác API có phụ thuộc trạng thái nhằm kiểm tra tính nhất quán của các quy tắc nghiệp vụ trong quá trình vận hành hệ thống.

Đầu vào của bài toán bao gồm tài liệu đặc tả OAS và địa chỉ truy cập của hệ thống REST API cần kiểm thử. Từ các thông tin này, hệ thống cần phân tích cấu trúc endpoint, tham số, kiểu dữ liệu và quan hệ phụ thuộc giữa các tài nguyên để sinh ra các kịch bản kiểm thử nhiều bước. Trong quá trình thực thi, các tác tử liên tục cập nhật trạng thái từ dữ liệu phản hồi nhằm điều chỉnh chiến lược khám phá và suy luận các điều kiện nghiệp vụ liên trạng thái.

Đầu ra của hệ thống là tập các báo cáo lỗi logic nghiệp vụ kèm theo chuỗi yêu cầu và dữ liệu thực thi tương ứng nhằm đảm bảo khả năng tái hiện lỗi. Bài toán đặt ra hai yêu cầu kỹ thuật chính. Thứ nhất, hệ thống phải đảm bảo các yêu cầu sinh ra tuân thủ các ràng buộc tĩnh được mô tả trong tài liệu OAS nhằm hạn chế các lỗi không hợp lệ ở mức giao thức hoặc cấu trúc dữ liệu. Thứ hai, hệ thống cần có khả năng suy luận và kiểm chứng các quy tắc nghiệp vụ không được đặc tả tường minh trong OAS, đặc biệt đối với các quan hệ phụ thuộc trạng thái giữa nhiều lời gọi API liên tiếp.

1.3 Mục tiêu và phạm vi nghiên cứu

Mục tiêu của khóa luận là xây dựng kiến trúc đa tác tử hỗ trợ tự động phát hiện lỗi logic nghiệp vụ trong hệ thống REST API có trạng thái với độ tin cậy cao. Hệ thống hướng tới việc giảm tỷ lệ dương tính giả trong quá trình kiểm thử, nâng cao khả năng suy luận các quy tắc nghiệp vụ liên trạng thái và cải thiện hiệu quả khám phá không gian kiểm thử thông qua việc kết hợp mô hình ngôn ngữ lớn với cơ chế bộ nhớ và chiến lược thực thi thích nghi.

Để đạt được mục tiêu trên, khóa luận tập trung vào ba hướng cải tiến kỹ thuật chính. Thứ nhất, xây dựng cơ chế khai thác ràng buộc tĩnh từ tài liệu OpenAPI nhằm kiểm tra tính hợp lệ của dữ liệu đầu vào trước khi thực hiện lời gọi API. Thứ hai, thiết kế kiến trúc bộ nhớ có cấu trúc kết hợp vòng lặp phản hồi chuyên gia nhằm hỗ trợ tác tử lưu trữ, truy xuất và hiệu chỉnh tri thức trong quá trình kiểm thử. Thứ ba, cải tiến chiến lược sinh và lựa chọn kịch bản kiểm thử nhằm cân bằng giữa việc khám phá các endpoint mới và khai thác sâu các vùng hành vi có khả năng chứa lỗi logic.

Đối tượng nghiên cứu được giới hạn trong các hệ thống REST API có trạng thái, trong đó các hành vi nghiệp vụ phụ thuộc vào chuỗi tương tác giữa nhiều lời gọi API liên tiếp. Về mặt kỹ thuật, hệ thống sử dụng LLM làm thành phần suy luận trung tâm và OAS làm nguồn tri thức cấu trúc đầu vào. Phạm vi nghiên cứu tập trung vào kiểm thử hộp đen đối với lỗi logic nghiệp vụ; các vấn đề liên quan đến hiệu năng hệ thống, kiểm thử tải và bảo mật hạ tầng không nằm trong phạm vi của khóa luận.

1.4 Cấu trúc khóa luận

Khóa luận được tổ chức thành 5 chương với nội dung chính như sau.

Chương 1 trình bày bối cảnh nghiên cứu, động lực thực hiện đề tài, bài toán đặt ra, mục tiêu nghiên cứu và phạm vi của khóa luận.

Chương 2 giới thiệu cơ sở lý thuyết liên quan đến REST API, đặc tả OpenAPI (OAS), mô hình ngôn ngữ lớn, kỹ thuật prompting, kiến trúc bộ nhớ Hindsight.

Chương 3 trình bày thiết kế và kiến trúc của hệ thống ARGOS. Chương này tập trung mô tả cơ chế khai thác ràng buộc tĩnh từ tài liệu OAS, kiến trúc bộ nhớ hỗ

trợ suy luận của tác tử và cơ chế tương tác người dùng.

Chương 4 trình bày quá trình cài đặt thực nghiệm, môi trường đánh giá và kết quả kiểm thử trên các hệ thống REST API thực tế. Các kết quả được phân tích dựa trên khả năng phát hiện lỗi logic nghiệp vụ và mức độ kiểm soát dương tính giả của hệ thống.

Cuối cùng, **Chương 5** tổng kết các kết quả đạt được, đánh giá những hạn chế còn tồn tại và đề xuất các hướng phát triển trong tương lai.

Chương 2

Cơ sở lý thuyết

2.1 Kiến trúc REST, REST API và Đặc tả OpenAPI

2.1.1 Kiến trúc REST

Kiến trúc REST (Representational State Transfer), được đề xuất bởi Roy Fielding [5], là một kiểu kiến trúc phổ biến trong xây dựng dịch vụ web. REST tổ chức hệ thống xoay quanh khái niệm tài nguyên (resource), trong đó mỗi tài nguyên được định danh thông qua URI và được thao tác bằng các phương thức HTTP chuẩn như *GET*, *POST*, *PUT* và *DELETE*.

Một trong những đặc điểm quan trọng của REST là tính phi trạng thái (*stateless*), nghĩa là mỗi yêu cầu từ phía máy khách phải chứa đầy đủ thông tin cần thiết để máy chủ xử lý mà không phụ thuộc vào trạng thái của các yêu cầu trước đó.

2.1.2 REST API

REST API (RESTful API) [6] được xây dựng trên nền của kiến trúc REST, được sử dụng rộng rãi trong việc phát triển các dịch vụ web thực tế. Các REST API thường trao đổi dữ liệu thông qua định dạng JSON và sử dụng mã trạng thái HTTP như 200 (OK), 201 (Created), 404 (Not Found) hay 500 (Internal Server Error) để chuẩn hóa phản hồi giữa máy khách và máy chủ.

Mặc dù giao thức REST mang tính phi trạng thái, logic nghiệp vụ của hệ thống lại thường có tính trạng thái. Việc thực thi thành công một phương thức POST để khởi tạo tài nguyên thường là điều kiện tiên đề bắt buộc để các yêu cầu GET hoặc DELETE tiếp theo hợp lệ. Ví dụ, trong một hệ thống quản lý sản phẩm, việc truy vấn thông tin sản phẩm qua GET /products/id chỉ có ý nghĩa nếu sản phẩm đó đã được tạo trước đó thông qua POST /products. Sự phụ thuộc tuần tự này tạo ra các chuỗi logic phức tạp, đòi hỏi quá trình kiểm thử phải duy trì và theo dõi được vòng đời trạng thái của hệ thống xuyên suốt chuỗi lời gọi API.

2.1.3 Đặc tả OpenAPI

OpenAPI Specification (OAS) là chuẩn mô tả giao diện REST API được sử dụng rộng rãi trong các hệ thống hiện đại. Tài liệu OAS định nghĩa các endpoint, phương thức HTTP, tham số đầu vào, cấu trúc dữ liệu và định dạng phản hồi của hệ thống. Việc khai thác tài liệu OAS cho phép hệ thống kiểm thử kiểm tra tính hợp lệ của yêu cầu đầu vào trước khi gửi tới máy chủ.

```
{
  "requestBody": {
    "description": "List of Genomic Locations",
    "content": {
      "application/json": {
        "schema": {
          "type": "array",
          "items": {
            "$ref": "#/components/schemas/GenomicLocation"
          }
        }
      }
    },
    "required": true
  },
}
```

Hình 2.1: Lược đồ yêu cầu của API `/annotation/genomic` trong OAS Genome Nexus

Xét ví dụ tại Hình 2.1 và Hình 2.2 về cấu trúc dữ liệu đầu vào của API truy vấn vị trí gen từ hệ thống Genome Nexus. Thông qua phân tích tài liệu OAS, hệ thống có thể chiết xuất các ràng buộc được mô tả tường minh, bao gồm kiểu dữ liệu, trường bắt buộc và cấu trúc của payload đầu vào. Ví dụ, endpoint `/annotation/genomic` yêu cầu dữ liệu đầu vào phải là một mảng các đối tượng *GenomicLocation*, trong đó các trường như *chromosome*, *start* và *end* phải tuân thủ đúng kiểu dữ liệu được định nghĩa. Việc tuân thủ tuyệt đối các ràng buộc này là điều kiện tiên quyết để

kịch bản kiểm thử vượt qua lớp xác thực bề mặt của máy chủ, tránh lỗi định dạng HTTP 400.

```
{
  "GenomicLocation": {
    "required": [
      "chromosome",
      "end",
      "start",
      ...
    ],
    "type": "object",
    "properties": {
      "chromosome": {
        "type": "string",
        "description": "Chromosome"
      },
      "start": {
        "type": "integer",
        "description": "Start Position",
        "format": "int32"
      },
      "end": {
        "type": "integer",
        "description": "End Position",
        "format": "int32"
      },
      ...
    }
  }
}
```

Hình 2.2: Lược đồ đối tượng Genomic Location trong OAS Genome Nexus

OAS đóng vai trò quan trọng trong mô tả API, nhưng còn hạn chế trong việc biểu diễn các ràng buộc logic nghiệp vụ liên trạng thái. Ví dụ, với lược đồ *GenomicLocation*, các bộ phân tích tự động có thể suy ra định dạng dữ liệu nhưng không thể trực tiếp chiết xuất các bất biến như giá trị không âm hoặc quan hệ $start \leq end$, do các quy tắc này thường chỉ xuất hiện dưới dạng mô tả phi cấu trúc hoặc không được khai báo. Các luồng tương tác tuần tự như yêu cầu tài nguyên phải tồn tại trước khi cập nhật cũng không được định nghĩa chính thức trong OAS. Vì vậy, OAS chưa đủ để tự động xây dựng đầy đủ các kịch bản kiểm thử logic phức tạp, đòi hỏi hệ thống phải bổ sung cơ chế suy luận động để khám phá các quy tắc nghiệp vụ ẩn.

2.2 Kiểm thử logic trong REST API

2.2.1 Kiểm thử mờ

Kiểm thử mờ là kỹ thuật kiểm thử tự động dựa trên việc sinh và gửi số lượng lớn dữ liệu đầu vào nhằm khám phá các hành vi bất thường của hệ thống. Trong bối cảnh REST API, fuzzing thường được triển khai theo hai hướng chính: mutation-based fuzzing và specification-based fuzzing sử dụng tài liệu OpenAPI để sinh dữ liệu kiểm thử. Các công cụ Fuzzing tiêu biểu như RESTest [7] hay Schemathesis [8] đã chứng minh được tính hiệu quả cao trong việc rà quét độ ổn định của máy chủ trước các dữ liệu dị thường.

2.2.2 Kiểm thử logic

Logic nghiệp vụ là tập hợp các quy tắc chi phối cách hệ thống xử lý và thay đổi dữ liệu theo yêu cầu của miền ứng dụng. Lỗi logic nghiệp vụ xảy ra khi hệ thống xử lý đúng về mặt kỹ thuật nhưng tạo ra trạng thái không phù hợp với quy tắc thực tiễn. Khác với lỗi hạ tầng, các lỗi này thường không kích hoạt mã lỗi 5xx mà vẫn trả về phản hồi thành công 2xx, dù trạng thái hệ thống đã vi phạm ràng buộc nghiệp vụ, chẳng hạn cho phép thanh toán với giỏ hàng có tổng giá trị âm.

Đặc điểm này khiến các kỹ thuật fuzzing truyền thống gặp hạn chế trong kiểm thử logic nghiệp vụ. Các phương pháp này chủ yếu tối ưu độ bao phủ mã nguồn thông qua sinh dữ liệu ngẫu nhiên thiếu ngữ nghĩa, nên khó mô hình hóa các quan hệ

phụ thuộc trạng thái giữa nhiều lời gọi API liên tiếp.

Vì vậy, kiểm thử logic nghiệp vụ không chỉ yêu cầu sinh các yêu cầu hợp lệ về mặt cấu trúc, mà còn cần khả năng duy trì ngữ cảnh và suy luận quy tắc nghiệp vụ thông qua tương tác thực tế với API.

2.3 Mô hình ngôn ngữ lớn

2.3.1 Mô hình ngôn ngữ lớn

Mô hình ngôn ngữ lớn (LLM) là các mô hình học sâu dựa trên kiến trúc Transformer [9], có khả năng xử lý và suy luận trên dữ liệu ngôn ngữ tự nhiên ở quy mô lớn [10]. Nhờ khả năng học trong ngữ cảnh (in-context learning), suy luận đa bước và gọi công cụ (tool calling), LLM có thể thực hiện các tác vụ phức tạp thông qua chỉ dẫn ngôn ngữ tự nhiên mà không cần huấn luyện chuyên biệt cho từng bài toán.

Trong kiểm thử REST API, các khả năng này cho phép LLM phân tích tài liệu OpenAPI, suy luận quan hệ phụ thuộc giữa các endpoint và xây dựng các chuỗi tương tác có trạng thái phù hợp với logic nghiệp vụ. Tuy nhiên, việc áp dụng LLM cho kiểm thử API vẫn tồn tại nhiều hạn chế liên quan đến khả năng duy trì ngữ cảnh dài hạn và độ tin cậy của quá trình suy luận.

2.3.2 Hạn chế của Mô hình ngôn ngữ lớn

Một trong những hạn chế quan trọng của LLM là giới hạn cửa sổ ngữ cảnh. Khi lượng thông tin đầu vào tăng lên, khả năng truy xuất và duy trì các chi tiết quan trọng của mô hình suy giảm đáng kể. Hiện tượng này thường được gọi là “lost in the middle” [11], khi mô hình có xu hướng ưu tiên các thông tin xuất hiện ở đầu và cuối ngữ cảnh, trong khi bỏ sót các nội dung nằm ở giữa.

Bên cạnh giới hạn ngữ cảnh, LLM còn dễ phát sinh hiện tượng ảo giác (hallucination) [12], tức đưa ra các kết luận có vẻ hợp lý về mặt ngôn ngữ nhưng không chính xác về mặt thực tế. Trong kiểm thử REST API, hiện tượng này có thể khiến tác tử suy luận sai các quy tắc nghiệp vụ hoặc đưa ra các giả định không phù hợp với hành vi thực tế của hệ thống.

2.3.3 Kỹ thuật Prompting

Prompting là phương pháp điều khiển hành vi LLM thông qua thiết kế câu lệnh đầu vào mà không cần huấn luyện lại mô hình.

Zero-shot và Few-shot prompting. Zero-shot prompting cung cấp chỉ dẫn trực tiếp không kèm ví dụ, phù hợp khi LLM đã có kiến thức nền đủ mạnh. Few-shot prompting bổ sung các ví dụ minh họa trong prompt để hướng dẫn mô hình định dạng đầu ra, đặc biệt hiệu quả khi cần tuân theo một mẫu cụ thể. Biến thể *dynamic few-shot* chọn ví dụ linh hoạt dựa trên ngữ cảnh hiện tại nhằm cải thiện độ phù hợp của prompt.

Chain-of-Thought (CoT) prompting [13] khuyến khích mô hình suy luận từng bước trước khi đưa ra câu trả lời. Ý tưởng chính là chèn các bước suy luận trung gian nhằm tách bài toán phức tạp thành các bài toán đơn giản hơn. Wei et al. [13] chứng minh rằng few-shot CoT cải thiện đáng kể hiệu suất trên các benchmark suy luận so với few-shot prompting thông thường.

ReAct (Reasoning + Acting) [14] kết hợp suy luận và hành động trong một vòng lặp: mô hình sinh ra suy nghĩ (*thought*), thực hiện hành động (*action*), quan sát kết quả (*observation*), rồi tiếp tục suy nghĩ và hành động tiếp theo. Cơ chế này cho phép mô hình điều chỉnh kế hoạch dựa trên phản hồi thực tế từ môi trường. ReAct đặc biệt phù hợp cho các tác vụ tương tác với môi trường bên ngoài, nơi kết quả hành động trước ảnh hưởng trực tiếp đến quyết định tiếp theo.

2.4 Kiến trúc Hệ đa tác tử dựa trên LLM

2.4.1 Kiến trúc Hệ đa tác tử

Tác tử dựa trên LLM (LLM-based Agent) là thực thể tự trị sử dụng LLM làm bộ não suy luận để quan sát môi trường, lập kế hoạch và thực thi hành động thông qua ngôn ngữ tự nhiên hoặc công cụ bên ngoài. Nhờ khả năng suy luận đa bước, duy trì ngữ cảnh và gọi công cụ, các tác tử LLM có thể xử lý những nhiệm vụ phức tạp đòi hỏi tương tác liên tục với hệ thống.

Hệ đa tác tử (Multi-Agent System – MAS) mở rộng mô hình này bằng cách tổ chức nhiều tác tử chuyên biệt phối hợp với nhau để giải quyết bài toán thông qua phân chia vai trò và trao đổi thông tin. Theo Wooldridge [15], trạng thái của một tác tử

có thể được biểu diễn bởi bộ ba $\langle P, A, G \rangle$, gồm tri thức quan sát được (*Perception*), tập hành động khả thi (*Actions*) và mục tiêu cục bộ (*Goals*).

AutoGen là framework hỗ trợ xây dựng hệ thống đa tác tử dựa trên LLM thông qua cơ chế hội thoại giữa các tác tử [16]. Trong AutoGen, mỗi tác tử được định nghĩa với vai trò, mục tiêu và khả năng riêng, đồng thời có thể trao đổi thông điệp với các tác tử khác để phối hợp giải quyết nhiệm vụ phức tạp. Framework hỗ trợ tích hợp công cụ bên ngoài, thực thi mã nguồn, quản lý lịch sử hội thoại và điều phối luồng tương tác giữa nhiều tác tử. Nhờ cơ chế giao tiếp dựa trên hội thoại, AutoGen cho phép xây dựng các quy trình suy luận nhiều bước, trong đó các tác tử có thể phản biện, xác minh hoặc bổ sung kết quả cho nhau trước khi đưa ra quyết định cuối cùng.

2.4.2 Bộ nhớ ngắn hạn và dài hạn

Trong các hệ thống tác tử sử dụng LLM, bộ nhớ thường được chia thành hai nhóm chính: bộ nhớ ngắn hạn và bộ nhớ dài hạn. Bộ nhớ ngắn hạn tương ứng với cửa sổ ngữ cảnh của mô hình, nơi lưu trữ thông tin phục vụ trực tiếp cho phiên suy luận hiện tại. Tuy nhiên, dung lượng của vùng nhớ này bị giới hạn và suy giảm hiệu quả khi số lượng thông tin đầu vào tăng lên.

Để giải quyết vấn đề đó, các hệ thống tác tử hiện đại thường sử dụng bộ nhớ dài hạn dưới dạng bộ nhớ ngoại vi. Khác với bộ nhớ ngắn hạn phụ thuộc vào cửa sổ ngữ cảnh, bộ nhớ ngoại vi cho phép lưu trữ thông tin trong các cơ sở dữ liệu vector, hỗ trợ truy xuất động khi cần.

Vòng lặp phản hồi cũng đóng vai trò quan trọng trong việc giảm ảnh hưởng của hiện tượng ảo giác từ LLM. Thay vì phụ thuộc hoàn toàn vào suy luận nội tại của mô hình, hệ thống liên tục đối chiếu các giả thuyết được sinh ra với phản hồi thực tế từ máy chủ. Nhờ đó, các quyết định lập kế hoạch và sinh dữ liệu kiểm thử dần được điều chỉnh dựa trên bằng chứng thực nghiệm thay vì chỉ dựa trên suy luận ngôn ngữ.

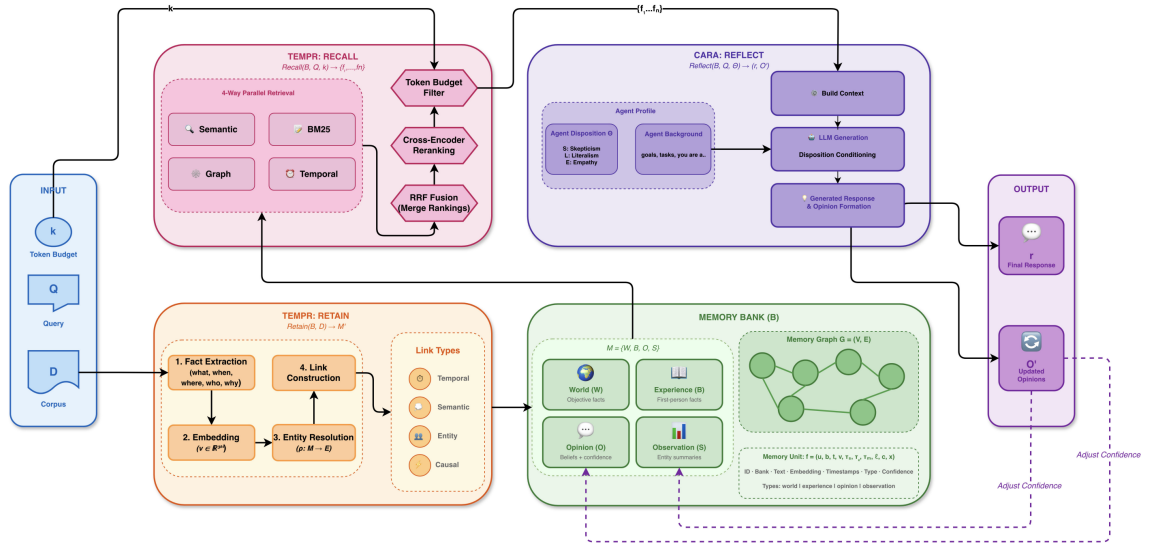
2.5 Kiến trúc bộ nhớ cấu trúc

2.5.1 Hạn chế của kiến trúc RAG

Các hệ thống tác tử hiện tại phần lớn dựa trên kỹ thuật RAG (Retrieval-Augmented Generation) truyền thống, nơi bộ nhớ chỉ được coi là một lớp lưu trữ bên ngoài dùng để trích xuất các đoạn văn bản rời rạc và đưa vào ngữ cảnh của một mô hình ngôn ngữ không trạng thái. Cách tiếp cận này bộc lộ nhiều hạn chế nghiêm trọng khi xử lý các tương tác dài hạn: hệ thống dễ bị pha loãng ngữ cảnh, không thể phân định rạch ròi giữa bằng chứng khách quan và niềm tin chủ quan, đồng thời thiếu tính nhất quán trong lập luận qua nhiều phiên làm việc.

2.5.2 Kiến trúc bộ nhớ Hindsight

Một trong những hướng tiếp cận tiêu biểu cho bài toán bộ nhớ dài hạn của tác tử tự trị là kiến trúc Hindsight [17]. Thay vì lưu toàn bộ lịch sử tương tác dưới dạng hội thoại phẳng, Hindsight tổ chức tri thức thành các đơn vị kinh nghiệm có cấu trúc, cho phép tác tử lưu trữ, truy xuất và cập nhật thông tin theo ngữ cảnh cụ thể. Hình 2.3 minh họa kiến trúc bộ nhớ của Hindsight với các thành phần lưu trữ tri thức và cơ chế tương tác giữa các pha lưu giữ (retain), truy xuất (recall) và phản tư (reflect).



Hình 2.3: Kiến trúc bộ nhớ Hindsight và các cơ chế Retain, Recall, Reflect

Trong Hindsight, mỗi tương tác với môi trường không chỉ bao gồm hành động và kết quả thực thi mà còn đi kèm với thông tin phản tư (reflection) được sinh ra sau quá trình quan sát phản hồi. Cơ chế này cho phép hệ thống lưu lại cả các trường hợp thành công lẫn thất bại để hỗ trợ quá trình lập kế hoạch trong các vòng lặp tiếp theo.

Kiến trúc này đặc biệt phù hợp với kiểm thử REST API có trạng thái do bản chất của bài toán đòi hỏi khả năng duy trì lịch sử tương tác dài hạn và khai thác lại các trạng thái đã quan sát trước đó. Thay vì phụ thuộc hoàn toàn vào ngữ cảnh tạm thời của LLM, tác tử có thể truy xuất lại các kinh nghiệm thực thi liên quan để: tái sử dụng các biến trạng thái động như token hoặc resource identifier; ghi nhận các chuỗi tương tác dẫn đến lỗi nghiệp vụ; lưu lại các mẫu phản hồi bất thường từ hệ thống; lập hồ sơ về các thực thể hỗ trợ sinh chiến lược kiểm thử cho các vòng lặp tiếp theo.

Thông qua cơ chế lưu vết và phản tư có cấu trúc, bộ nhớ Hindsight đóng vai trò nền tảng hỗ trợ tác tử duy trì nhận thức trạng thái dài hạn, giảm phụ thuộc vào cửa sổ ngữ cảnh của LLM và cải thiện chất lượng suy luận trong quá trình kiểm thử logic tự động.

2.6 Các nghiên cứu liên quan

2.6.1 LogiAgent: Hệ đa tác tử dựa trên LLM cho kiểm thử logic REST API

LogiAgent là một trong những nghiên cứu tiên phong áp dụng hệ đa tác tử dựa trên LLM cho bài toán kiểm thử logic nghiệp vụ REST API. Công trình này cho thấy LLM không chỉ có khả năng sinh dữ liệu ngẫu nhiên mà còn có thể suy luận chuỗi tương tác API dựa trên ngữ cảnh nghiệp vụ và quan hệ phụ thuộc giữa các endpoint.

Kiến trúc của LogiAgent được xây dựng xoay quanh ba thành phần chính: tác tử sinh kịch bản kiểm thử, tác tử thực thi yêu cầu và tác tử thẩm định phản hồi. Hệ thống sử dụng API Relationship Graph (ARG) để biểu diễn quan hệ ngữ nghĩa giữa các endpoint và hỗ trợ sinh chuỗi lời gọi API có tính liên kết. Bên cạnh đó, Execution Memory được sử dụng để lưu trữ lịch sử thực thi nhằm hỗ trợ tái sử dụng các giá trị trạng thái đã quan sát trước đó.

Kết quả thực nghiệm cho thấy LogiAgent có khả năng phát hiện số lượng lớn lỗi logic nghiệp vụ trên nhiều hệ thống REST API khác nhau. Tuy nhiên, nghiên cứu này cũng bộc lộ hai hạn chế quan trọng. Thứ nhất, hệ thống chưa có cơ chế xác thực ràng buộc tĩnh từ đặc tả OpenAPI trước khi thực thi, dẫn đến việc nhiều yêu cầu vi phạm định dạng hoặc tham số bắt buộc bị trả về lỗi 4xx nhưng vẫn bị tác tử diễn giải nhầm thành lỗi logic nghiệp vụ. Thứ hai, Execution Memory được tổ chức dưới dạng lưu vết phẳng (unstructured execution memory), chưa hỗ trợ phân tách các loại tri thức khác nhau như sự kiện thực thi, giả thuyết suy luận và phản hồi xác thực từ hệ thống. Điều này làm gia tăng nguy cơ tích lũy suy luận sai lệch khi quy mô kiểm thử tăng lên.

2.6.2 RBCTest: Khai phá và xác minh oracle phản hồi API REST bằng mô hình ngôn ngữ lớn

RBCTest tập trung giải quyết bài toán suy luận ràng buộc kiểm thử từ tài liệu OpenAPI nhằm cải thiện chất lượng oracle và giảm hiện tượng ảo giác của LLM trong quá trình sinh kiểm thử. Điểm đóng góp chính của nghiên cứu là cơ chế Observation-Confirmation (OC), trong đó quá trình suy luận được chia thành hai bước liên tiếp: quan sát ràng buộc tiềm năng và xác nhận lại tính hợp lệ của ràng buộc đó trước khi sử dụng cho kiểm thử.

Cách tiếp cận này cho thấy việc phân tách suy luận thành nhiều bước nhỏ giúp LLM giảm đáng kể các suy luận thiếu căn cứ khi diễn giải mô tả API. Thay vì trực tiếp sinh kiểm thử từ mô tả văn bản, hệ thống yêu cầu mô hình tự đánh giá lại các giả thuyết đã sinh ra trước khi đưa chúng vào quá trình thực thi.

Kết quả thực nghiệm của RBCTest cho thấy cơ chế xác nhận nhiều bước giúp cải thiện đáng kể độ chính xác khi chiết xuất ràng buộc từ OpenAPI. Nghiên cứu này cung cấp một hướng tiếp cận hiệu quả cho bài toán xác thực ràng buộc tĩnh — một hạn chế quan trọng còn tồn tại trong LogiAgent.

2.6.3 Hindsight: Kiến trúc bộ nhớ cho tác tử học như con người

Hindsight là kiến trúc bộ nhớ được đề xuất cho các tác tử AI hoạt động dài hạn trong môi trường động. Khác với cơ chế lưu lịch sử hội thoại dạng phẳng, Hindsight tổ chức tri thức thành các đơn vị kinh nghiệm có cấu trúc nhằm hỗ trợ lưu trữ,

truy xuất và cập nhật thông tin theo ngữ cảnh cụ thể.

Kiến trúc này tập trung giải quyết ba vấn đề chính của tác tử LLM: giới hạn bộ nhớ dài hạn, sự pha trộn giữa quan sát thực tế và suy luận chủ quan, và khả năng cập nhật tri thức theo phản hồi từ môi trường. Để giải quyết các vấn đề này, Hindsight phân tách thông tin thành nhiều lớp dữ liệu khác nhau, bao gồm sự kiện thực thi, lịch sử tương tác, quan sát và ý kiến suy luận. Bên cạnh đó, hệ thống hỗ trợ cơ chế reflection nhằm lưu lại cả các trường hợp thành công lẫn thất bại để phục vụ cho quá trình lập kế hoạch ở các vòng lặp tiếp theo.

Các kết quả thực nghiệm cho thấy Hindsight cải thiện đáng kể khả năng duy trì tính nhất quán và truy xuất thông tin dài hạn của tác tử. Đối với bài toán kiểm thử REST API có trạng thái, kiến trúc này đặc biệt phù hợp do hệ thống cần duy trì lịch sử lời gọi API, trạng thái động và các mẫu lỗi nghiệp vụ xuyên suốt nhiều vòng thực thi.

2.6.4 Khoảng trống nghiên cứu

Mặc dù LogiAgent chứng minh hiệu quả của hệ đa tác tử trong kiểm thử logic REST API, hệ thống vẫn tồn tại hai hạn chế chính: (i) thiếu cơ chế xác thực ràng buộc tĩnh từ đặc tả OpenAPI trước khi thực thi; và (ii) kiến trúc bộ nhớ chưa hỗ trợ phân tách và cập nhật tri thức có cấu trúc trong quá trình suy luận dài hạn.

Trong khi đó, RBCTest cho thấy cơ chế xác nhận nhiều bước có khả năng giảm hiện tượng ảo giác khi suy luận ràng buộc từ OpenAPI, còn Hindsight cung cấp kiến trúc bộ nhớ dài hạn phù hợp cho các tác tử tự trị hoạt động trong môi trường động.

Từ các nền tảng này, nghiên cứu đề xuất hướng tiếp cận kết hợp: kiến trúc đa tác tử cho kiểm thử logic từ LogiAgent; cơ chế xác thực ràng buộc tĩnh từ RBCTest; và kiến trúc bộ nhớ dài hạn của Hindsight.

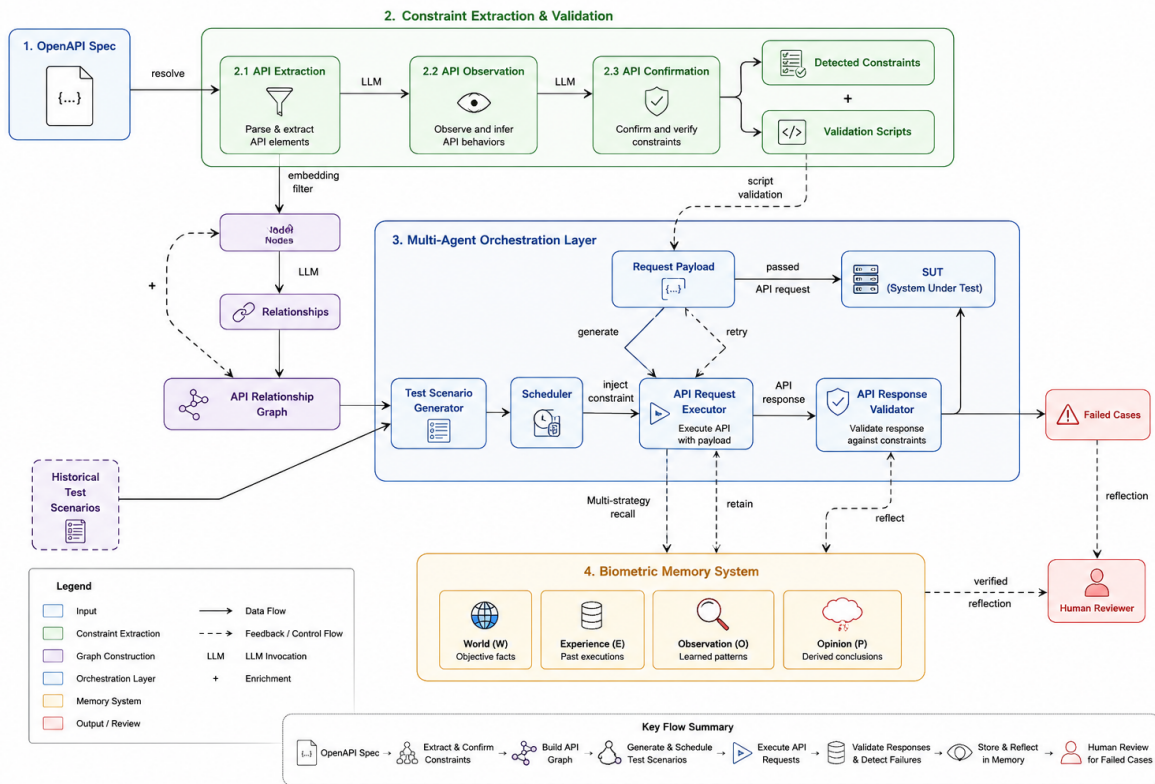
Thông qua sự kết hợp này, nghiên cứu hướng tới xây dựng một hệ thống kiểm thử REST API có khả năng suy luận logic, duy trì trạng thái dài hạn và tự điều chỉnh chiến lược kiểm thử dựa trên phản hồi thực tế từ hệ thống.

Chương 3

Phương pháp đề xuất

3.1 Tổng quan phương pháp đề xuất

Kiểm thử logic đối với REST API có trạng thái đòi hỏi khả năng suy luận liên trạng thái, duy trì lịch sử thực thi và khai thác các ràng buộc nghiệp vụ không được mô tả đầy đủ trong tài liệu OpenAPI. Để giải quyết các thách thức này, nghiên cứu đề xuất ARGOS (Adaptive Retrieval-Guided Orchestration System) — hệ thống kiểm thử logic REST API dựa trên kiến trúc đa tác tử kết hợp cơ chế xác thực ràng buộc tĩnh và bộ nhớ dài hạn cho tác tử tự trị.



Hình 3.1: Kiến trúc tổng thể của hệ thống ARGOS

ARGOS được thiết kế dựa trên bốn nguyên lý chính: (i) phân rã nhiệm vụ thông qua kiến trúc đa tác tử nhằm giảm tải ngữ cảnh; (ii) xác thực ràng buộc tĩnh trước thực thi để hạn chế sinh yêu cầu không hợp lệ; (iii) duy trì bộ nhớ dài hạn có cấu

trúc cho lịch sử thực thi và phản tư; (iv) sử dụng vòng lặp phản hồi để xác thực các phản tư của tác tử trước khi cập nhật vào bộ nhớ dài hạn.

Trong quy trình hoạt động của ARGOS, tài liệu OpenAPI trước tiên được phân tích để xây dựng API Relationship Graph (ARG) và trích xuất các ràng buộc tĩnh. Từ các ràng buộc này, hệ thống sử dụng cơ chế Observation-Confirmation để sinh ra các script kiểm chứng dưới dạng hàm Python nhằm xác thực payload trước khi gửi yêu cầu thực tế đến hệ thống cần kiểm thử. Sau đó, bộ điều phối kịch bản sử dụng thông tin từ ARG kết hợp với bộ nhớ Hindsight để lựa chọn endpoint và sinh chuỗi tương tác kiểm thử phù hợp.

Trong quá trình thực thi, các tác tử thực hiện sinh yêu cầu, gửi truy vấn HTTP, phân tích phản hồi và đánh giá kết quả dựa trên cả ràng buộc tĩnh lẫn ngữ cảnh trạng thái động. Các phản hồi thành công hoặc thất bại đều được lưu lại dưới dạng kinh nghiệm có cấu trúc trong bộ nhớ Hindsight nhằm hỗ trợ cho các vòng suy luận tiếp theo. Đối với các trường hợp bị đánh dấu là lỗi logic tiềm năng, hệ thống cho phép human reviewer xác thực lại phản tư trước khi hợp nhất vào bộ nhớ dài hạn nhằm hạn chế việc củng cố các giả thuyết sai lệch.

Các thành phần và cơ chế hoạt động chi tiết của ARGOS sẽ được trình bày trong các mục tiếp theo, bao gồm kiến trúc đa tác tử, cơ chế Observation-Confirmation, tích hợp bộ nhớ Hindsight và chiến lược khám phá kịch bản kiểm thử.

3.2 Kiến trúc kiểm thử kế thừa từ LogiAgent

3.2.1 API Relationship Graph

Trong kiểm thử REST API có trạng thái, các endpoint thường tồn tại quan hệ phụ thuộc về dữ liệu và luồng nghiệp vụ. Một yêu cầu có thể cần sử dụng đầu ra của yêu cầu trước đó làm tham số đầu vào, ví dụ như sử dụng `userId` được sinh từ API tạo người dùng để tiếp tục thực hiện các thao tác truy vấn hoặc cập nhật. Vì vậy, quá trình kiểm thử cần xây dựng được các chuỗi tương tác hợp lệ thay vì chỉ thực thi các endpoint độc lập.

Để biểu diễn các quan hệ này, LogiAgent đề xuất API Relationship Graph (ARG) [4], trong đó mỗi đỉnh biểu diễn một endpoint và mỗi cạnh biểu diễn mối quan hệ ngữ nghĩa giữa hai API. ARG đóng vai trò là không gian định hướng cho quá trình

sinh kịch bản kiểm thử nhiều bước.

Trong ARGOS, kiến trúc ARG được kế thừa trực tiếp từ LogiAgent. Quá trình xây dựng ARG bao gồm hai giai đoạn. Đầu tiên, embedding của các endpoint được sử dụng để loại bỏ các cặp API có độ tương đồng ngữ nghĩa thấp nhằm giảm chi phí suy luận. Sau đó, các cặp endpoint còn lại được đưa vào LLM để xác nhận quan hệ ngữ nghĩa và phụ thuộc nghiệp vụ giữa chúng. Những cặp được xác nhận sẽ được biểu diễn thành các cạnh trong đồ thị.

ARG được sử dụng làm nền tảng cho quá trình khám phá kịch bản kiểm thử. Thay vì lựa chọn endpoint hoàn toàn ngẫu nhiên, hệ thống thực hiện random walk trên đồ thị để xây dựng các chuỗi lời gọi API có tính liên kết nghiệp vụ và khả năng duy trì trạng thái giữa các bước thực thi.

3.2.2 Luồng hoạt động đa tác tử

Dựa trên kiến trúc đa tác tử của LogiAgent, ARGOS tổ chức quy trình kiểm thử thành nhiều tác tử chuyên biệt nhằm giảm tải ngữ cảnh và phân tách trách nhiệm suy luận của LLM. Mỗi tác tử chỉ xử lý một tập nhiệm vụ cụ thể thay vì phải duy trì toàn bộ trạng thái của quá trình kiểm thử trong cùng một ngữ cảnh.

Quá trình kiểm thử bắt đầu từ việc lựa chọn endpoint trên API Relationship Graph. Hệ thống sử dụng chiến lược ϵ -greedy để cân bằng giữa hai hành vi: khám phá các chuỗi API mới và khai thác các khu vực có lịch sử phát sinh lỗi. Trong pha khai thác, hệ thống ưu tiên các endpoint liên quan đến các phản tư và hotspot lỗi được truy xuất từ Hindsight. Trong pha khám phá, các endpoint được lấy mẫu thông qua random walk trên ARG nhằm mở rộng phạm vi kiểm thử.

Sau khi lựa chọn được chuỗi endpoint, Test Scenario Generator sinh kịch bản kiểm thử dựa trên OAS, trạng thái thực thi hiện tại và thông tin được truy xuất từ Hindsight. Kịch bản sinh ra sau đó được chuyển tới Test Scenario Recorder để cấu trúc hóa và lưu trữ thông tin thực thi trung gian.

Trước khi gửi yêu cầu HTTP tới hệ thống cần kiểm thử, payload được kiểm tra thông qua hàm kiểm chứng Python sinh ra từ cơ chế OC nhằm phát hiện các vi phạm ràng buộc tĩnh từ OAS. Các hàm kiểm chứng được sinh động bằng LLM và lưu trữ trong cơ sở dữ liệu để tái sử dụng trong các lần kiểm thử tiếp theo.

Sau bước xác thực payload, API Invoke Agent thực hiện gửi yêu cầu tới hệ thống

cần kiểm thử và chuyển phản hồi tới Validator Agent. Tác tử này đánh giá phản hồi dựa trên ngữ cảnh thực thi và suy luận từ các lỗi trong quá khứ. Khi phát hiện phản hồi bất thường hoặc sai lệch nghiệp vụ, Validator Agent sinh phản tư mô tả nguyên nhân tiềm năng của lỗi.

Các phản tư liên quan đến lỗi không được đưa trực tiếp vào bộ nhớ dài hạn mà phải trải qua bước xác thực thủ công trước khi cập nhật vào Hindsight. Cơ chế này nhằm hạn chế việc lưu trữ các suy luận sai hoặc hiện tượng ảo giác từ LLM trong quá trình kiểm thử lâu dài.

3.3 Cơ chế Observation–Confirmation cho xác thực ràng buộc tĩnh

Tài liệu OAS thường chứa nhiều ràng buộc được mô tả dưới dạng ngôn ngữ tự nhiên thay vì biểu diễn hình thức. Việc suy luận trực tiếp các ràng buộc này bằng LLM dễ dẫn đến hiện tượng ảo giác logic, khiến hệ thống sinh ra payload hoặc oracle kiểm thử không chính xác. Để giảm thiểu vấn đề này, ARGOS áp dụng cơ chế Observation–Confirmation kế thừa từ RBCTest nhằm tách quá trình suy luận thành hai bước độc lập. Trước tiên, LLM sinh các quan sát tự do từ thông tin OAS; sau đó, hệ thống thực hiện bước xác nhận để kiểm chứng tính hợp lệ và khả năng kiểm thử của các quan sát này trước khi chuyển chúng thành ràng buộc có cấu trúc. Các ràng buộc đã xác nhận tiếp tục được chuyển thành các verification script có thể thực thi nhằm xác thực payload trước khi gửi yêu cầu tới hệ thống đích.

3.3.1 Pha Observation

Trong pha Observation, hệ thống trích xuất thông tin mô tả của từng tham số hoặc thuộc tính từ tài liệu OAS, bao gồm tên tham số, kiểu dữ liệu, mô tả tự nhiên, schema và các ví dụ minh họa nếu có. Tập thông tin này được đưa vào prompt đầu tiên nhằm yêu cầu LLM phân tích và đưa ra các nhận định tự do về các ràng buộc tiềm ẩn.

Đầu ra của pha này là các observation ở dạng văn bản tự nhiên thay vì các luật kiểm thử cứng. Ví dụ, từ mô tả của tham số `isoformOverrideSource`, mô hình có thể sinh ra các observation như:

The parameter is optional. The value should be a string. The description suggests semantic values such as “uniprot”.

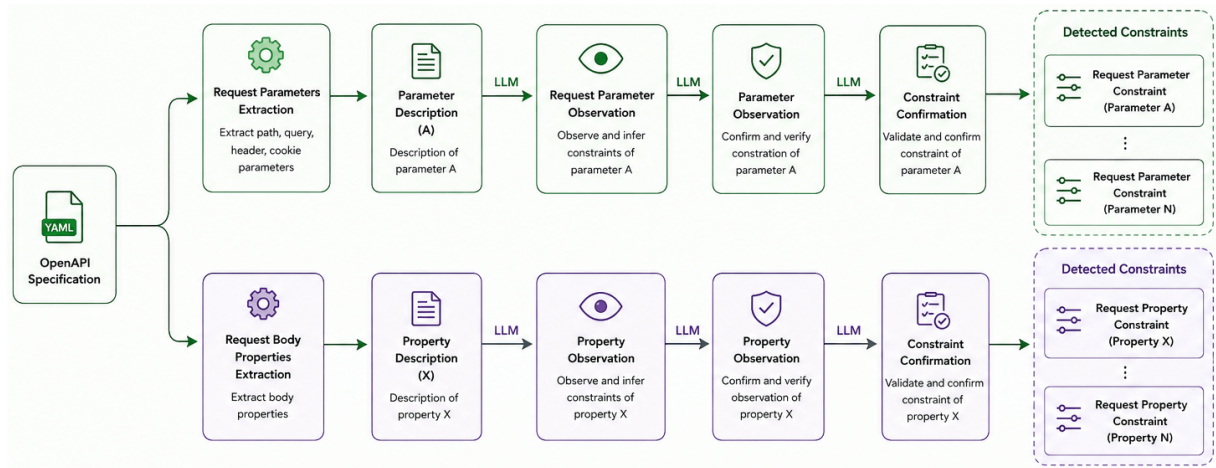
Các observation này có thể bao gồm: (i) Ràng buộc kiểu dữ liệu; (ii) Điều kiện bắt buộc hoặc tùy chọn; (iii) Quan hệ phụ thuộc ngữ nghĩa; (iv) Gợi ý về miền giá trị hoặc quy tắc nghiệp vụ. Tại giai đoạn này, hệ thống ưu tiên khả năng bao phủ tri thức hơn tính chính xác tuyệt đối. Do đó, observation chỉ đóng vai trò như các giả thuyết suy luận ban đầu và chưa được sử dụng trực tiếp cho quá trình kiểm thử.

3.3.2 Pha Confirmation

Các observation sinh ra từ bước trước tiếp tục được đưa vào một prompt xác nhận độc lập nhằm đánh giá: (i) liệu observation có thực sự được hỗ trợ bởi OAS; (ii) liệu ràng buộc có thể được kiểm chứng bằng thực thi hay không; (iii) và mức độ tin cậy của ràng buộc được suy luận. Mục tiêu của pha Confirmation là loại bỏ các suy luận mơ hồ hoặc mang tính suy diễn quá mức của LLM. Ví dụ, observation “parameter should be a known source” chỉ được giữ lại như một gợi ý nghiệp vụ mềm, trong khi observation “must be string” có thể được xác nhận thành một ràng buộc schema chính thức.

Sau khi xác nhận, các ràng buộc được chuẩn hóa thành cấu trúc dữ liệu gồm: loại ràng buộc; tham số nguồn; thuộc tính mục tiêu; mô tả tự nhiên; trạng thái xác nhận; điểm tin cậy. Các ràng buộc đã xác nhận được lưu trữ trong cơ sở dữ liệu để phục vụ tái sử dụng ở các vòng kiểm thử sau. Hệ thống cũng áp dụng cơ chế cache nhằm tránh thực hiện lại quá trình suy luận đối với các endpoint đã được phân tích trước đó và đảm bảo sự nhất quán.

Hình 3.2 minh họa quy trình Observation–Confirmation trong ARGOS.



Hình 3.2: Quy trình Observation–Confirmation trong ARGOS

3.3.3 Sinh và xác thực verification script

Sau khi ràng buộc được xác nhận, ARGOS tiếp tục sử dụng LLM để sinh ra các verification script dưới dạng hàm Python có khả năng thực thi trực tiếp. Mỗi script đóng vai trò như một executable oracle dùng để kiểm tra tính hợp lệ của payload hoặc phản hồi tương ứng với ràng buộc đã khai phá.

Ví dụ, đối với ràng buộc yêu cầu tham số `isoformOverrideSource` phải là chuỗi ký tự hợp lệ, hệ thống có thể sinh script kiểm chứng như sau:

```

1 def verify_latest_response(latest_response, request_info):
2     """
3     Validate the 'isoformOverrideSource' query parameter according to the
4     OpenAPI spec.
5     Returns:
6         1 - request is valid (parameter absent or present and a string)
7         -1 - request violates the specification (present but not a string)
8         0 - insufficient information to decide (e.g., request_info structure
9             missing)
10    """
11    # Guard against completely missing request_info
12    if not isinstance(request_info, dict):
13        return 0
14
15    # Query parameters may be stored under 'params' or directly under 'query'
16    query = request_info.get('params') or request_info.get('query') or {}
  
```

```

16     # If the key is not present, the field is optional -> valid (no violation)
17     if 'isoformOverrideSource' not in query:
18         return 1
19
20     # Retrieve the raw value (it may be URL-encoded)
21     raw_value = query.get('isoformOverrideSource')
22
23     # Decode URL-encoded strings safely; if it's already a plain string, this is
24     # a no-op
25     try:
26         from urllib.parse import unquote_plus
27         value = unquote_plus(str(raw_value))
28     except Exception:
29         # If decoding fails, treat it as a validation failure
30         return -1
31
32     # Type check: must be a string after decoding
33     if isinstance(value, str):
34         return 1
35     else:
36         return -1

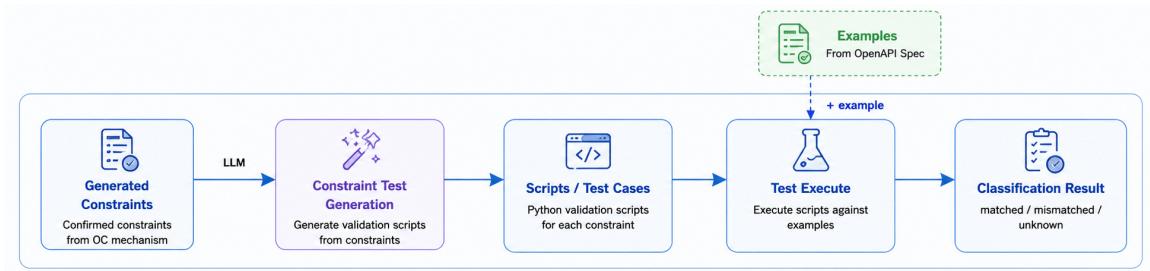
```

Hình 3.1: Ví dụ verification script được sinh tự động

Trong đó:

- Giá trị 1 biểu thị ràng buộc được thỏa mãn;
- Giá trị -1 biểu thị phát hiện vi phạm;
- Giá trị 0 biểu thị không đủ thông tin để xác minh.

Để giảm nguy cơ sinh oracle không chính xác, các script sau khi được tạo sẽ tiếp tục trải qua bước xác thực bằng ví dụ. Hệ thống thực thi script trên các example được định nghĩa trong OAS như hình 3.3; nếu script không vượt qua được các ví dụ tham chiếu, ràng buộc tương ứng sẽ bị loại bỏ thay vì đưa vào pipeline kiểm thử. Cơ chế này giúp hạn chế việc lưu trữ các oracle bị ảo giác và tăng độ tin cậy của tập ràng buộc được sử dụng trong quá trình kiểm thử tự động.



Hình 3.3: Quy trình kiểm chứng script với ví dụ

3.3.4 Tích hợp vào vòng kiểm thử

Các script được tích hợp vào vòng kiểm thử dưới dạng bước xác thực tiền thực thi. Sau khi Test Scenario Generator sinh payload cho một yêu cầu API, hệ thống sẽ thực thi các script tương ứng để kiểm tra tính hợp lệ của payload trước khi gửi yêu cầu thực tế tới hệ thống đích. Nếu payload vi phạm một hoặc nhiều ràng buộc tĩnh, yêu cầu sẽ bị loại bỏ và hệ thống quay lại bước sinh dữ liệu kiểm thử thay vì tiếp tục thực thi API. Điều này giúp hạn chế đáng kể các lỗi 4xx phát sinh do payload không hợp lệ, đồng thời giảm nguy cơ hệ thống nhầm lẫn giữa lỗi đặc tả và lỗi logic nghiệp vụ.

Khác với các phương pháp hậu kiểm chỉ kiểm tra phản hồi sau khi API đã được gọi, cơ chế của ARGOS tập trung phát hiện và loại bỏ các yêu cầu sai ngay từ trước giai đoạn thực thi. Nhờ đó, hệ thống có thể tập trung tài nguyên kiểm thử vào các tương tác có ý nghĩa nghiệp vụ thay vì tiêu tốn số lượng lớn lời gọi API cho các trường hợp vi phạm ràng buộc hiển nhiên.

3.4 Cơ chế tương tác người dùng (Human-in-the-loop)

Mặc dù ARGOS hướng tới tự động hóa quá trình kiểm thử API, hệ thống vẫn duy trì cơ chế human-in-the-loop nhằm kiểm soát chất lượng các giả thuyết lỗi được sinh bởi LLM. Hệ thống yêu cầu người vận hành xác minh các reflection quan trọng trước khi chúng được promote thành mental model chính thức trong Hindsight thay vì cho phép tác tử tự động cập nhật toàn bộ tri thức vào bộ nhớ dài hạn,

Giao diện tương tác người dùng được thể hiện dưới dạng giao diện dòng lệnh (CLI). Người dùng khởi chạy quá trình kiểm thử thông qua các tham số cấu hình như tài

liệu OAS, mô hình LLM sử dụng, SUT, và cấu hình các thành phần. Trong quá trình thực thi, hệ thống liên tục hiển thị trạng thái của các API đang được kiểm thử, kết quả xác thực và các reflection do validator agent sinh ra. Khi validator agent phát hiện một phản hồi nghi ngờ chứa lỗi nghiệp vụ, hệ thống sẽ hiển thị reflection tương ứng để người vận hành đánh giá. Một ví dụ tương tác được minh họa như sau:

```
[3/7] model_id=69fd7de8805ac0602bc06bc7
endpoint      : PUT /news/{id}
issue         : invalid_country_causes_500
details       : Invalid country causes 500 instead of 400.
Add validation to return 400 for invalid country values.
source        : agent_reflection
effective_run  : 2
```

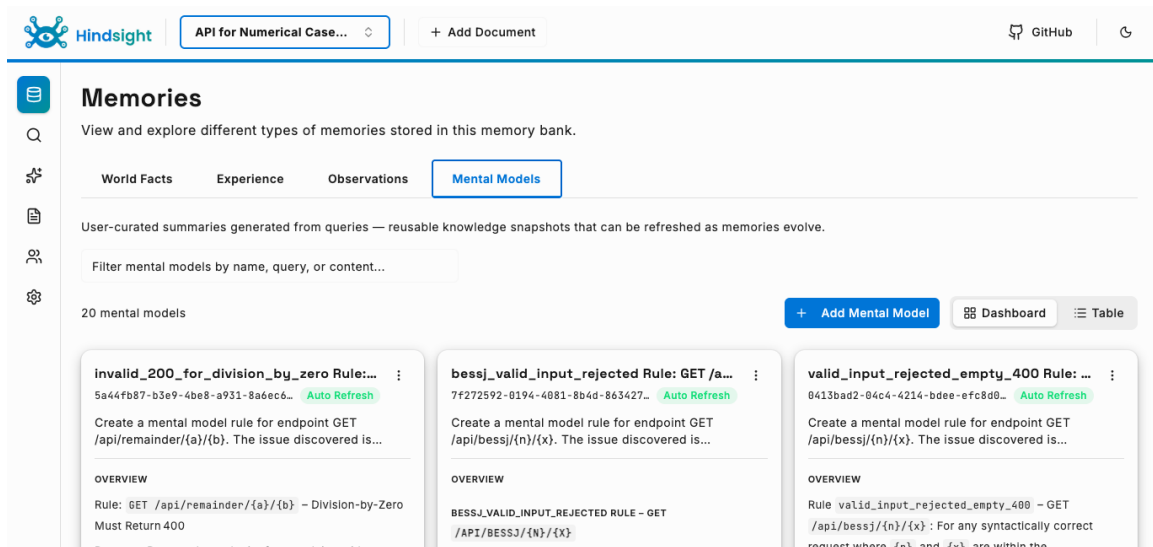
Actions: [Y] approve, [E] edit+approve, [N] reject,
[S] skip, [Q] quit

Người dùng có thể lựa chọn:

- **approve**: chấp nhận reflection và promote thành rule chính thức;
- **edit+approve**: chỉnh sửa reflection trước khi xác nhận;
- **reject**: loại bỏ giả thuyết lỗi;
- **skip**: bỏ qua mà không cập nhật bộ nhớ;
- **quit**: dừng phiên kiểm thử hiện tại.

Nếu reflection được phê duyệt, hệ thống sẽ gọi cơ chế **create_rule** để chuyển giả thuyết hiện tại thành tri thức dài hạn trong Opinion bank của Hindsight. Các rule đã được xác minh này sẽ tham gia vào quá trình Recall ở các vòng kiểm thử tiếp theo, cho phép ARGOS ưu tiên khai thác các vùng đã từng xuất hiện hành vi bất thường.

Cơ chế human-in-the-loop giúp hạn chế hiện tượng tích lũy tri thức sai do hallucination của LLM. Hệ thống duy trì sự phân tách giữa các sự kiện đã quan sát được



Hình 3.4: Giao diện control plane của Hindsight

(Observation); và các giả thuyết suy luận (Opinion) thay vì coi mọi reflection là sự thật tuyệt đối. Chỉ các reflection được xác minh bởi người vận hành mới được promote thành mental model có độ tin cậy cao. Điều này giúp bộ nhớ dài hạn của hệ thống duy trì được tính ổn định và khả năng truy vết trong các phiên kiểm thử kéo dài.

Bên cạnh giao diện CLI, ARGOS còn sử dụng control plane của Hindsight (hình 3.4) để theo dõi trạng thái bộ nhớ, các execution episodes và các rule đã được promote trong quá trình kiểm thử.

3.5 Tích hợp kiến trúc bộ nhớ Hindsight

Để hỗ trợ kiểm thử REST API có trạng thái trong các phiên thực thi kéo dài, ARGOS tích hợp kiến trúc bộ nhớ Hindsight như một tầng bộ nhớ dài hạn cho hệ đa tác tử. Thay vì lưu toàn bộ lịch sử thực thi dưới dạng hội thoại phẳng hoặc nhật ký tuần tự, Hindsight tổ chức tri thức thành nhiều phân vùng bộ nhớ với vai trò nhận thức luận khác nhau. Cách tiếp cận này cho phép hệ thống duy trì đồng thời tri thức từ đặc tả API, lịch sử tương tác thực tế và các phản tư được sinh ra trong quá trình kiểm thử.

Trong ARGOS, tầng bộ nhớ này đóng vai trò như một cơ chế học tập liên tục cho tác tử kiểm thử. Sau mỗi vòng thực thi, hệ thống không chỉ lưu request và response mà còn trích xuất các mẫu hành vi, tổng hợp quan sát và ghi nhận các

giả thuyết lỗi để phục vụ cho các vòng kiểm thử tiếp theo. Các tri thức này sau đó được truy xuất thông qua cơ chế multi-strategy recall nhằm hỗ trợ sinh kịch bản, tạo payload và xác thực phản hồi.

3.5.1 Ánh xạ tri thức kiểm thử vào Hindsight

ARGOS ánh xạ các loại tri thức trong bài toán kiểm thử REST API vào bốn phân vùng bộ nhớ của Hindsight bao gồm: World, Experience, Observation và Opinion. Việc phân tách này giúp hệ thống tránh trộn lẫn giữa tri thức đặc tả, dữ liệu thực nghiệm và các giả thuyết do tác tử suy luận.

Bảng 3.1: Ánh xạ tri thức kiểm thử REST API vào các phân vùng bộ nhớ Hindsight

Memory Bank	Loại tri thức	Ví dụ trong ARGOS
World (W)	Tri thức khách quan từ đặc tả API	Ánh xạ endpoint-path, kiểu dữ liệu request/response
Experience (E)	Lịch sử thực thi và tương tác	Request-response logs, payload đã gửi, mã trạng thái HTTP, execution traces
Observation (S)	Quan sát trung lập rút ra từ thực thi	Endpoint profile, authentication requirements, execution summaries
Opinion (O)	Giả thuyết lỗi và phản tư của tác tử	Hotspot về lỗi, anomaly patterns, verified reflections

Trong đó, World lưu các tri thức có tính khách quan và tương đối ổn định được trích xuất từ tài liệu OAS. Các tri thức này bao gồm kiểu dữ liệu, tham số bắt buộc và cấu trúc response. Đây được xem là “nguồn chân lý” của hệ thống và được sử dụng để xác thực payload trước thực thi.

Experience lưu các phiên tương tác thực tế với hệ thống đích, bao gồm payload đã gửi, response nhận được và các chuỗi lời gọi API theo thời gian. Phân vùng này đóng vai trò như bộ nhớ hồi ức của tác tử, cho phép hệ thống truy xuất lại các trạng thái đã quan sát trong các phiên kiểm thử trước đó.

Observation lưu các tổng hợp trung lập được rút ra từ Experience. Thay vì lưu toàn bộ lịch sử thực thi thô, ARGOS sinh ra các execution summaries để mô tả hành vi của endpoint dưới dạng cô đọng hơn, ví dụ: “Endpoint này thường trả về lỗi 400 khi thiếu trường authentication”. Các quan sát này không được xem là chân lý tuyệt đối mà chỉ phản ánh xu hướng hành vi được quan sát trong thực tế.

Opinion lưu các giả thuyết lỗi và phản tư được sinh ra bởi Validator Agent khi phát hiện các phản hồi bất thường. Mỗi reflection được biểu diễn dưới dạng bán cấu trúc, bao gồm các trường như `focus`, `anomaly_pattern`, `reasoning` và `confidence`. Những phản tư chưa được xác thực sẽ chỉ tồn tại dưới dạng hypothesis. Chỉ các phản tư đã được xác minh bởi chuyên gia mới được nâng cấp thành quy luật chính thức trong mental model của hệ thống.

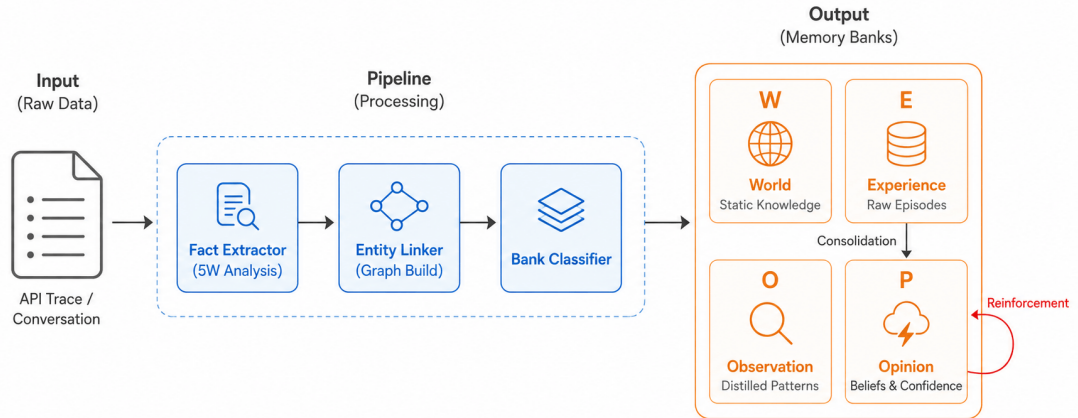
Cơ chế phân vùng này giúp ARGOS tránh việc đồng nhất các giả thuyết suy luận với tri thức đặc tả chính thức. Nhờ đó, hệ thống có khả năng duy trì ranh giới rõ ràng giữa: tri thức khách quan từ OAS; quan sát thực nghiệm từ quá trình thực thi; và các niềm tin hoặc giả thuyết do tác tử suy luận. Điều này đặc biệt quan trọng trong môi trường kiểm thử REST API có trạng thái, nơi các phản hồi bất thường có thể xuất phát từ lỗi nghiệp vụ thực sự, nhưng cũng có thể chỉ là hệ quả của trạng thái tạm thời hoặc payload không hợp lệ.

3.5.2 Cơ chế Retain

Hình 3.5 mô tả quy trình lưu trữ tri thức trong kiến trúc Hindsight được tích hợp trong ARGOS. Thay vì trực tiếp lưu toàn bộ lịch sử tương tác dưới dạng hội thoại phẳng, hệ thống thực hiện một chuỗi xử lý nhằm chuyển đổi dữ liệu thực thi thô thành các đơn vị tri thức có cấu trúc trước khi phân bổ vào các memory banks tương ứng.

Ở đầu vào, hệ thống tiếp nhận các dữ liệu thực thi bao gồm request/response traces, lịch sử hội thoại giữa các tác tử và thông tin ngữ cảnh phát sinh trong quá trình kiểm thử. Các dữ liệu này trước tiên được đưa qua tầng xử lý trung gian

gồm ba bước chính.



Hình 3.5: Quy trình Retain trong ARGOS

Trong bước *Fact Extraction*, hệ thống trích xuất các thực thể và sự kiện quan trọng từ dữ liệu thực thi theo mô hình phân tích ngữ cảnh dạng “5W” như endpoint được gọi, tham số đầu vào, mã phản hồi, thời điểm thực thi hoặc trạng thái xác thực. Tiếp theo, bước *Entity Linking* xây dựng các liên kết ngữ nghĩa giữa các thực thể nhằm hình thành cấu trúc quan hệ phục vụ truy xuất sau này. Cuối cùng, bước *Bank Classification* phân loại thông tin vào các nhóm bộ nhớ phù hợp dựa trên vai trò ngữ nghĩa của dữ liệu.

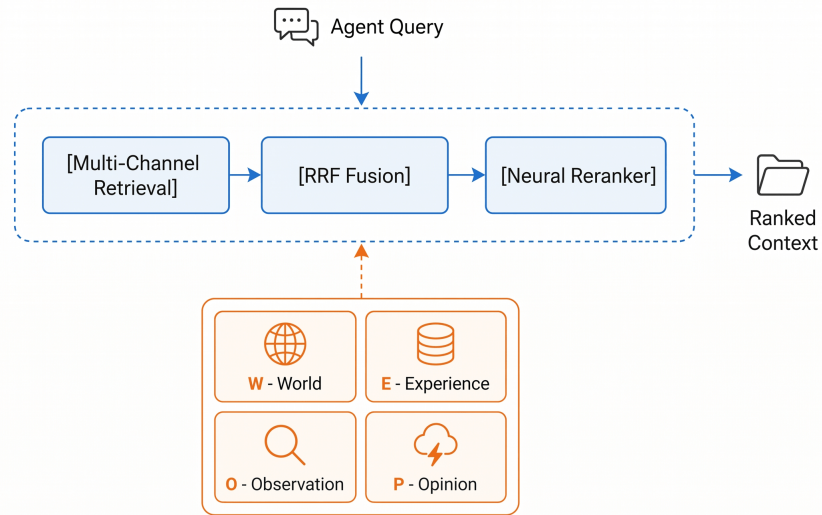
Sau quá trình xử lý, tri thức được phân bổ vào bốn memory banks của Hindsight. Trong đó, World (W) lưu trữ tri thức tĩnh được suy ra trực tiếp từ tài liệu OAS như endpoint mappings, schema hoặc các ràng buộc cấu trúc. Experience (E) lưu các episode thực thi thô bao gồm request/response logs và trạng thái tương tác trong từng phiên kiểm thử. Từ các episode này, hệ thống thực hiện quá trình *Consolidation* nhằm tổng hợp các mẫu hành vi lặp lại hoặc các đặc điểm quan sát được để hình thành Observation (O). Khác với Experience vốn chỉ lưu dữ liệu nguyên bản, Observation biểu diễn các mẫu hành vi đã được cô đọng ở mức khái quát hơn, ví dụ như “endpoint thường trả về mã 401 khi thiếu token xác thực”.

Trên cơ sở các observations và phản hồi từ quá trình xác thực lỗi, hệ thống hình thành Opinion (P) — tầng bộ nhớ biểu diễn các giả thuyết, nhận định hoặc niềm tin của tác tử về hành vi bất thường của hệ thống. Các opinion này có thể được

củng cố hoặc điều chỉnh thông qua cơ chế reinforcement dựa trên kết quả thực thi mới hoặc xác nhận từ con người. Cách tổ chức phân tầng này giúp ARGOS duy trì ranh giới rõ ràng giữa tri thức khách quan và các suy luận mang tính giả thuyết, qua đó hạn chế hiện tượng nhiễu tri thức và hỗ trợ tác tử học tập dài hạn trong quá trình kiểm thử REST API có trạng thái.

3.5.3 Cơ chế Recall

Trong quá trình sinh kịch bản và thực thi kiểm thử, ARGOS liên tục truy xuất tri thức từ Hindsight thông qua cơ chế Recall nhằm cung cấp cho tác tử các ngữ cảnh thực thi liên quan thay vì phụ thuộc hoàn toàn vào cửa sổ ngữ cảnh ngắn hạn của LLM. Kiến trúc Recall được minh họa trong Hình 3.6.



Hình 3.6: Cơ chế multi-strategy Recall trong ARGOS

Khác với cơ chế truy xuất đơn lẻ chỉ dựa trên semantic similarity, ARGOS áp dụng chiến lược multi-channel retrieval kết hợp nhiều tín hiệu truy xuất khác nhau từ các memory banks của Hindsight, bao gồm World, Experience, Observation và Opinion. Đối với mỗi truy vấn từ tác tử, hệ thống thực hiện song song nhiều phương pháp truy xuất như semantic retrieval dựa trên embedding similarity, lexical retrieval dựa trên BM25 và graph-based retrieval dựa trên quan hệ giữa các endpoint trong API Relationship Graph. Ngoài ra, hệ thống cũng ưu tiên các execution episodes gần nhất thông qua cơ chế temporal reranking. Các kết quả truy xuất từ nhiều

kênh sau đó được hợp nhất bằng thuật toán Reciprocal Rank Fusion (RRF). Cơ chế này giúp giảm phụ thuộc vào một phương pháp truy xuất đơn lẻ và tăng khả năng giữ lại các ngữ cảnh có mức độ liên quan cao xuất hiện đồng thời ở nhiều kênh khác nhau. Sau bước hợp nhất, hệ thống sử dụng neural reranker để đánh giá lại mức độ liên quan của từng ngữ cảnh đối với truy vấn hiện tại trước khi tạo ra tập ranked context cuối cùng cho tác tử.

Cơ chế Recall được sử dụng tại nhiều giai đoạn của hệ thống, bao gồm sinh kịch bản kiểm thử, sinh payload yêu cầu và xác thực phản hồi trong validator agent. Trong quá trình này, tác tử có thể truy xuất lại các execution episodes trước đó, các endpoint gần đây, các anomaly patterns đã quan sát hoặc các hotspot từng xuất hiện lỗi để hỗ trợ quá trình suy luận và ra quyết định.

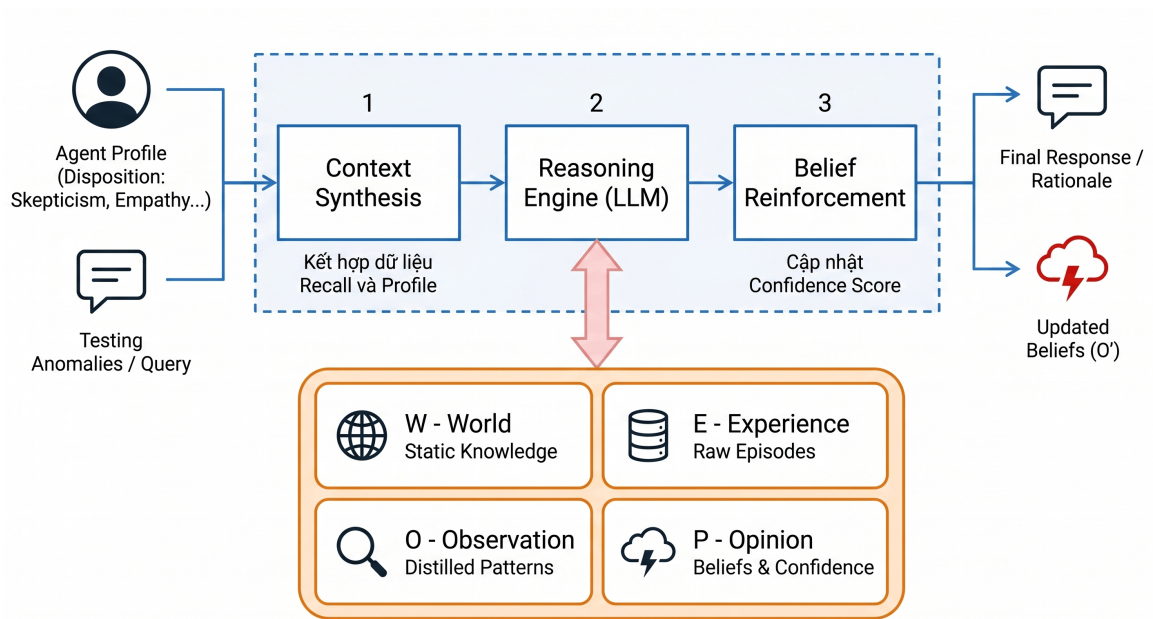
Nhờ kiến trúc multi-strategy Recall, ARGOS có khả năng duy trì tính liên tục của tri thức giữa nhiều vòng kiểm thử và giảm phụ thuộc vào giới hạn ngữ cảnh của mô hình ngôn ngữ lớn.

3.5.4 Reflection và cập nhật tri thức

Khi validator agent phát hiện phản hồi nghi ngờ chứa lỗi nghiệp vụ, ARGOS kích hoạt cơ chế Reflection để hình thành các giả thuyết về nguyên nhân và mẫu hành vi bất thường của hệ thống.

Quy trình Reflection được minh họa trong Hình 3.7. Đầu vào của quá trình này bao gồm: các testing anomalies được phát hiện trong phiên thực thi hiện tại; ngữ cảnh được truy xuất từ Hindsight thông qua cơ chế Recall; và profile hành vi của tác tử kiểm thử.

Trong bước đầu tiên, hệ thống thực hiện *context synthesis* bằng cách kết hợp thông tin từ các memory banks gồm World, Experience, Observation và Opinion để tạo ngữ cảnh suy luận. Ngữ cảnh này sau đó được đưa vào reasoning engine dựa trên LLM nhằm sinh ra các phản tư (*reflection*) về hành vi bất thường của hệ thống. Khác với việc chỉ đánh dấu request là "pass" hoặc fail", reflection cung cấp thêm lập luận về nguyên nhân tiềm năng của lỗi. Các reflection được biểu diễn dưới dạng bán cấu trúc nhằm hỗ trợ lưu trữ và truy xuất về sau, thường bao gồm: anomaly pattern được quan sát; reasoning của tác tử; và confidence score tương ứng.



Hình 3.7: Quy trình Reflection và cập nhật tri thức trong ARGOS

Ví dụ, sau nhiều lần thực thi thất bại, validator có thể sinh reflection:

“Endpoint PUT /users thường trả về lỗi 400 khi thiếu username mặc dù không required trong đặc tả”

Các reflection này ban đầu được lưu dưới dạng giả thuyết trong Opinion và chưa được xem là tri thức xác thực. Sau khi được review bởi chuyên gia, các reflection hợp lệ có thể được promote thành mental models chính thức với độ tin cậy cao hơn. Ngược lại, các giả thuyết không được xác nhận sẽ không được sử dụng để củng cố belief state của hệ thống.

Cơ chế Reflection giúp ARGOS tích lũy tri thức phản tư theo thời gian thay vì chỉ lưu trữ execution logs thô. Đồng thời, việc tách biệt giữa observation khách quan và opinion suy luận cũng giúp hạn chế sự lan truyền của các giả thuyết sai do LLM sinh ra.

3.6 Phân tích ưu điểm và hạn chế

3.6.1 Ưu điểm

ARGOS kế thừa kiến trúc kiểm thử logic đa tác tử từ LogiAgent đồng thời mở rộng bằng cơ chế xác thực ràng buộc tĩnh và bộ nhớ dài hạn có cấu trúc. Sự kết

hợp này mang lại một số ưu điểm quan trọng đối với bài toán kiểm thử REST API có trạng thái.

Giảm tỷ lệ dương tính giả trong quá trình kiểm thử logic. Nhờ áp dụng cơ chế OC để trích xuất và xác thực các ràng buộc tĩnh từ tài liệu OAS trước giai đoạn thực thi, các payload không hợp lệ bị loại bỏ trước khi gửi request thực tế, giúp hạn chế các trường hợp lỗi 4xx bị nhầm lẫn với lỗi nghiệp vụ của hệ thống. Bên cạnh đó, phương pháp cải thiện khả năng duy trì tri thức dài hạn thông qua kiến trúc bộ nhớ Hindsight. Nhờ đó, hệ thống có thể duy trì ngữ cảnh kiểm thử trong thời gian dài mà không phụ thuộc hoàn toàn vào cửa sổ ngữ cảnh ngắn hạn của LLM.

Học từ kinh nghiệm theo thời gian Các phản tư từ validator agent không được cập nhật trực tiếp thành tri thức chính thức mà phải trải qua bước xác minh của người vận hành trước khi được promote thành mental model. Điều này giúp tăng độ tin cậy của bộ nhớ dài hạn và giảm nguy cơ tích lũy các suy luận không chính xác theo thời gian.

Linh hoạt về kiến trúc Kiến trúc phân tách thành nhiều tác tử chuyên biệt cho phép thay thế và nâng cấp các thành phần độc lập, đồng thời cũng giúp giảm tải ngữ cảnh cho từng LLM agent.

3.6.2 Hạn chế

Mặc dù đạt được nhiều cải tiến so với các kiến trúc kiểm thử truyền thống, ARGOS vẫn tồn tại một số hạn chế nhất định.

Phụ thuộc vào chất lượng suy luận của LLM Hầu hết các tác vụ đều sử dụng LLM làm thành phần trung tâm và yêu cầu khả năng suy luận ở mức tương đối. Khi mô hình sinh ra suy luận thiếu chính xác hoặc bị ảo giác logic, hệ thống vẫn có thể tạo ra các giả thuyết sai hoặc bỏ sót lỗi tiềm ẩn.

Phụ thuộc vào độ tin cậy của tài liệu OAS Trong thực tế, nhiều hệ thống REST API không duy trì OAS đồng bộ với cài đặt hiện tại. Các schema lỗi thời, thiếu endpoint hoặc mô tả không đầy đủ có thể khiến quá trình trích xuất ràng buộc trở nên thiếu chính xác, từ đó ảnh hưởng trực tiếp tới cơ chế OC và validation scripts.

Chi phí tính toán và token tương đối cao ARGOS sử dụng nhiều lời gọi LLM

trong một vòng kiểm thử. Ngoài ra, việc duy trì nhiều tầng lưu trữ như Neo4j cho ARG, MongoDB cho constraints/scripts và Hindsight memory trên PostgreSQL kết hợp pgvector cũng làm tăng độ phức tạp triển khai của hệ thống.

Giảm mức độ tự động Cơ chế human-in-the-loop giúp tăng độ tin cậy của mental model nhưng đồng thời cũng làm giảm mức độ tự động hóa hoàn toàn của hệ thống. Trong các môi trường kiểm thử quy mô lớn với số lượng reflection lớn, quá trình xác minh thủ công có thể trở thành điểm nghẽn vận hành nếu không có cơ chế ưu tiên hoặc lọc phản tư hiệu quả.

Chương 4

Thực nghiệm và đánh giá

4.1 Mục tiêu thực nghiệm

Chương này trình bày quá trình thực nghiệm và đánh giá hệ thống ARGOS trên tập các REST API thực tế nhằm kiểm chứng hiệu quả của các cải tiến được đề xuất trong khóa luận. Thực nghiệm tập trung đánh giá khả năng phát hiện lỗi logic nghiệp vụ, kiểm soát dương tính giả, hiệu quả của từng thành phần trong kiến trúc và mức độ bao phủ kiểm thử đạt được.

Các kết quả thực nghiệm được đối chiếu với LogiAgent – kiến trúc nền tảng mà nghiên cứu kế thừa và mở rộng. Nghiên cứu tập trung trả lời bốn câu hỏi nghiên cứu chính sau đây:

- **RQ1:** Kiến trúc đề xuất có cải thiện khả năng phát hiện lỗi logic nghiệp vụ so với LogiAgent gốc không?
- **RQ2:** Cơ chế Observation–Confirmation kết hợp bộ nhớ Hindsight có giúp giảm tỷ lệ dương tính giả so với LogiAgent gốc không?
- **RQ3:** Mỗi thành phần cải tiến trong ARGOS đóng góp như thế nào vào hiệu quả tổng thể của hệ thống?
- **RQ4:** ARGOS đạt được độ bao phủ thao tác và mã nguồn ở mức nào so với các công cụ kiểm thử REST API hiện có?

4.2 Thiết lập thực nghiệm

4.2.1 Dịch vụ thực nghiệm

Thực nghiệm được tiến hành trên tổng cộng 10 hệ thống REST API như bảng 4.1, bao gồm: 01 dịch vụ trực tuyến công khai và 09 dịch vụ triển khai cục bộ trên môi trường Docker.

Bảng 4.1: Các dịch vụ sử dụng cho đánh giá

Dịch vụ	Số dòng code	Số API
Features Service	1,646	18
Rest Countries	32,494	22
News Service	590	7
SCS	634	11
NCS	569	6
Language Tool	677,521	2
Person Controller	1,386	12
Project Track	88,634	67
User Management	5,108	22
Genome Nexus (Online)	Online	23

Các hệ thống được lựa chọn là các dịch vụ được xây dựng bằng ngôn ngữ Java, bao phủ nhiều miền nghiệp vụ khác nhau, được sử dụng từ bộ dữ liệu thực nghiệm của LogiAgent. Mỗi hệ thống đều cung cấp tài liệu đặc tả OAS phục vụ cho quá trình sinh kịch bản kiểm thử và khai phá ràng buộc.

Để đảm bảo tính khách quan trong quá trình đối chiếu, nghiên cứu này sử dụng tập lỗi logic đã được xác nhận từ LogiAgent làm tập dữ liệu chuẩn và thực hiện với ngân sách 500 yêu cầu API trên mỗi dịch vụ.

4.2.2 Môi trường thực nghiệm

Trong quá trình thực nghiệm, hệ thống sử dụng nhiều backbone LLM khác nhau, chủ yếu là GPT-OSS-120B và Tencent Hy3 Preview thông qua LiteLLM Proxy nhằm cân bằng giữa chất lượng sinh suy luận và chi phí thực thi. Mô hình embedding sử dụng Gemma Embedding được self-host để phục vụ semantic retrieval và xây dựng API Relation Graph (ARG).

Các dịch vụ REST API được triển khai thông qua Docker trên máy cục bộ sử dụng MacBook Pro Intel Core i7 4-core với 16GB RAM. Các thành phần lưu trữ và bộ nhớ dài hạn được triển khai trên nền tảng DBaaS cloud nhằm giảm tải tài nguyên cục bộ và hỗ trợ truy xuất trạng thái ổn định trong quá trình thực nghiệm dài

hạn. Cấu hình chi tiết của môi trường thực nghiệm được trình bày trong Bảng 4.2.

Bảng 4.2: Môi trường thực nghiệm của ARGOS

Thành phần	Cấu hình
Máy chủ	MacBook Pro Intel Core i7 (4-core), 16GB RAM
Hệ điều hành	macOS
Container runtime	Docker
LLM backbone	GPT-OSS-120B, Tencent Hy3 Preview
LLM gateway	LiteLLM Proxy + PostgreSQL (Aiven DBaaS)
Mô hình embedding	Gemma Embedding (self-hosted)
Đồ thị ARG	Neo4j cloud
Ràng buộc & script kiểm chứng	MongoDB cloud
Bộ nhớ Hindsight	PostgreSQL + pgvector + TimescaleDB (Aiven DBaaS)
Prompt caching	Redis
SUT	Docker-based REST services + Online APIs

4.3 Phương pháp đối sánh

Để đánh giá hiệu quả của kiến trúc đề xuất, khoá luận sử dụng LogiAgent làm hệ thống đối sánh chính. Việc đánh giá được thực hiện theo phương pháp so sánh cắt giảm, trong đó các thành phần được bổ sung tuần tự vào LogiAgent nhằm phân tích mức đóng góp của từng cải tiến đối với hiệu quả tổng thể của hệ thống. Các cấu hình thực nghiệm được trình bày trong Bảng ??.

Mỗi cấu hình được thực thi độc lập trên cùng tập REST services với cùng ngân sách request nhằm bảo đảm tính công bằng trong đánh giá. Trước mỗi lần chạy, toàn bộ execution logs, cache và trạng thái bộ nhớ ngắn hạn đều được khởi tạo lại nhằm loại bỏ ảnh hưởng của các lần thực nghiệm trước.

4.4 Kết quả thực nghiệm

4.4.1 RQ1: Khả năng phát hiện lỗi

Bảng 4.3: Thống kê số issue

Phân loại	Số lượng	Tỉ lệ (%)
Bug	139	41.87
Bug nhỏ	136	40.96
False-Positive Bugs	57	17.17
Tổng số	332	100%

Đối với RQ1, hệ thống được đánh giá dựa trên số lượng lỗi logic nghiệp vụ được phát hiện trên từng REST service. Một issue được xem là hợp lệ khi thỏa mãn các điều kiện:

- phản hồi bất thường có thể tái hiện ổn định;
- vi phạm kỳ vọng nghiệp vụ hoặc trạng thái HTTP;
- và được xác nhận thông qua ground truth hoặc kiểm tra thủ công.

Các issue được phân loại thành ba nhóm:

- Bug: lỗi nghiệp vụ thực sự;
- Enhancement: thiếu validation hoặc xử lý chưa phù hợp;
- False Positive: báo cáo sai do hallucination.

Các độ đo chính bao gồm:

- tổng số issue được phát hiện;
- số lượng bug hợp lệ;
- tỷ lệ bug trên tổng số report;
- và tỷ lệ false-positive.

4.4.2 RQ2: Khả năng kiểm soát False positive

Đối với RQ2, khoá luận đánh giá khả năng kiểm soát false-positive

Nhóm độ đo này phản ánh mức độ hiệu quả của tầng Observation và Hindsight trong việc hạn chế hiện tượng hallucination khi suy luận ràng buộc bằng LLM.

4.4.3 RQ3: Đóng góp của từng thành phần

Bảng 4.4: Tỷ lệ FP trung bình của mỗi phương pháp

Cấu hình	FP trung bình (%)
Baseline	32.82
Cơ chế OC	32.94
HITL	32.21
Bộ nhớ Hindsight	24.16
ARGOS	17.17

Đối với RQ3, khoá luận sử dụng phương pháp thực nghiệm cắt giảm nhằm đánh giá ảnh hưởng riêng của từng thành phần:

- Observation–Confirmation;
- Hindsight;
- và Human-in-the-loop.

Mức đóng góp của mỗi thành phần được đánh giá dựa trên sự thay đổi của:

- số lượng bug được phát hiện;
- tỷ lệ false-positive;
- và độ bao phủ endpoint.

4.4.4 RQ4: Độ bao phủ kiểm thử

Đối với RQ4, luận văn sử dụng hai nhóm độ đo coverage.

Operation Coverage Operation coverage được định nghĩa là tỷ lệ operation trong OAS đã được thực thi ít nhất một lần:

$$\text{Coverage}_{op} = \frac{\# \text{ executed operations}}{\# \text{ total operations}}$$

Code Coverage Đối với các hệ thống có mã nguồn khả dụng, khoá luận sử dụng JaCoCo để đo:

- line coverage;
- branch coverage;
- và method coverage.

4.5 Phân tích và thảo luận

Kết luận

Tóm tắt bài toán, giải pháp và các kết quả đạt được

Tài liệu tham khảo

- [1] Viettel IDC. 200 http status code là gì?, 2025.
- [2] Traceable. 2025 global state of api security report, 2025.
- [3] Salt Security. Owasp api security top 10: Insights from the api security trenches, 2023. Accessed: 2026-03-14.
- [4] Ke Zhang, Chenxi Zhang, Chong Wang, Chi Zhang, YaChen Wu, Zhenchang Xing, Yang Liu, Qingshan Li, and Xin Peng. Logiagent: Automated logical testing for rest systems with llm-based multi-agents, 2025.
- [5] Roy Thomas Fielding. *Architectural Styles and the Design of Network-based Software Architectures*. PhD thesis, University of California, Irvine, 2000.
- [6] Adeel Ehsan, M. A. Abuhaliqa, C. Catal, and Deepti Mishra. Restful api testing methodologies: Rationale, challenges, and solution directions. *Applied Sciences*, 12(9):4369, 2022.
- [7] Alberto Martin-Lopez, Sergio Segura, and Antonio Ruiz-Cortés. Resttest: Automated black-box testing of restful web apis. In *Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA '21)*, pages 682–685. ACM, 2021.
- [8] Zac Hatfield-Dodds and Dmitry Dygalo. Deriving semantics-aware fuzzers from web api schemas, 2021.
- [9] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [10] Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, Yifan Du, Chen Yang, Yushuo Chen, Zhipeng Chen, Jinhao Jiang, Ruiyang Ren, Yifan Li, Xinyu Tang, Zikang Liu, Peiyu Liu, Jian-Yun Nie, and Ji-Rong Wen. A survey of large language models, 2023.

- [11] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts, 2023.
- [12] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Yejin Bang, Andrea Madotto, and Pascale Fung. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12):1–38, 2023.
- [13] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [14] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models, 2022.
- [15] Michael Wooldridge. *An Introduction to MultiAgent Systems*. John Wiley & Sons, Chichester, UK, 2 edition, 2009.
- [16] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryen W. White, Doug Burger, and Chi Wang. Autogen: Enabling next-gen llm applications via multi-agent conversation. In *arXiv preprint arXiv:2308.08155*, 2023.
- [17] Chris Latimer, Nicolás Boschi, Andrew Neeser, Chris Bartholomew, Gaurav Srivastava, Xuan Wang, and Naren Ramakrishnan. Hindsight is 20/20: Building agent memory that retains, recalls, and reflects, 2025.